
Ablation Study for Parameter-Golf Challenge

Antonio Honsell	Andrea Lamonarca	Giulio Latora	Matteo Mello Grand	Fabian Menekshi	Kevin Shaqiri
3199937	3243056	3259119	3245806	3241043	3246712

GitHub Code Repository

Abstract

We tackle a variant of OpenAI’s *Parameter Golf Challenge*, in which a GPT language model (LM) must be trained under strict time and memory constraints in order to minimise its Bit-per-Byte (BpB) loss on a held-out validation set of the FineWeb dataset. In our specific setting, training is restricted to 5000 iterations on a single NVIDIA A100 GPU and the final compressed artifact must fit within 16 MB. We improve over the OpenAI’s *Naive Baseline* along two directions. On the architecture side, we introduce 4 low-parameter-cost mechanisms (3 from leaderboard and 1 novel) that enrich the model’s signal routing, and ablate each individually as well as in tandem. On the quantization side, we analyse the performance–memory trade-off of more aggressive compression and adopt a mixed-bitwidth scheme combined with quantization-aware training. Our final model reaches 1.2812 BpB with 15.36 MB, a 0.029 BpB improvement with smaller compressed size.

1 Introduction

The Parameter-Golf Challenge asks for a GPT model under a 16 MB compressed-artifact cap and a 10-minute wall-clock training budget on 8×H100. Our setting deliberately differs from the official challenge in 2 main ways: training is measured in *iterations* (5000) rather than wall-clock time, and runs are on a single A100 GPU rather than 8 H100. The single-GPU constraint reflects the Bocconi’s HPC hardware limitations while the iteration-based budget is a methodological choice. Under a fixed step count every variant receives the same number of gradient updates regardless of its per-step cost, so observed differences in validation loss are attributable to the technique itself rather than to compute-speed differences.

Aligned with the original challenge, the main metric we use to evaluate and compare different models is the bits per byte (BPB) validation loss computed on a held-out FineWeb validation split. BPB is a tokenizer-agnostic compression measure derived from the next-token cross-entropy normalised by the byte length of the underlying text. Lower BPB therefore corresponds to better compression and, equivalently, better predictive modelling of the corpus.

2 Background and Related Work

In this section we will introduce and discuss briefly the main features of the baseline model as well as the record submissions to the *Parameter-Golf Challenge* from which we have taken inspiration to perform our study and analysis.

Baseline Model As displayed in diagrams 3 and 4 in appendix, the OpenAI baseline is a GPT model with tied embeddings (same matrix E is used for embedding and de-embedding), a stack of $L = 9$ pre-normalised transformer blocks, hidden dimension of $d = 512$ and a sequence length $T = 1024$. It differs from standard GPT models mainly because it facilitates signal propagation over depth by enforcing a U-net structure of weighted skip connections between the first L_{enc} encoder blocks and the remaining decoder blocks as well as by utilizing learnable *residual mixing* of current and initial hidden state at the beginning of each block. The final embeddings are also soft-capped to avoid extreme output scores in the usual next-token Cross-Entropy (CE) Loss. Thus, parameters are optimized for CE with Muon and Adam optimizers depending on their type, C.5. The resulting model weights are then quantized only post-training to INT8 in symmetric fashion and further compressed with zlib. Therefore, the quantized baseline model occupies 15.78 MB and reaches a final val_bpb of 1.3101 ± 0.0013 . Full architecture, optimiser configuration, and quantization details are in Appendix C.2.

Sources for Quantization We use uniform INT k symmetric weight-only quantization as the baseline, and compare against GPTQ Frantar et al. [2023], AWQ Lin et al. [2026], and NF4 Dettmers et al. [2023] as more advanced post-training methods, plus quantization-aware training (QAT) Jacob et al. [2017] as the training-side alternative. We also try to replicate the most

popular quantization approach used in the leaderboard, which is based on GPTQ and LQER Zhang et al. [2024]. (Details in Appendix C.14).

Sources for Architecture Design The three main architectural additions we adopt over the baseline are each inspired by a specific Parameter Golf leaderboard submission. The two-lane *Parallel Residuals* and the mid-stack *Depth Recurrence* formulations both come from the PR + DR submission of harborglowvintage oss [2026], which independently converged on the same mid-stack two-pass configuration (layers 3–5) we identified in our own sweeps. The per-head *Smeared Attention Output Gate* follows the record submission by MarioPaerle [2026]. Each mechanism is grounded in prior academic work — PaLM-style parallel blocks [Chowdhery et al., 2023] for PR, Universal Transformers [Dehghani et al., 2019] and ALBERT [Lan et al., 2020] for DR, and gated attention [Qiu et al., 2025] for the gate — but the specific adaptations we use most closely follow the two leaderboard submissions above. Also, we try to incorporate the more flexible approach of Attention Residuals from the recent paper Kimi et al. [2026] with the Baseline’s fixed skip connections and residual mixing streams.

3 Architecture Method and Experiments

In this section, we analyse the experiments and ablations performed on 4 different architectural additions to the baseline model: *Parallel Residuals* (PR), *Depth Recurrence* (DR), per-head *Attention Output Gate* and *Attention Residuals*. Therefore, we implement each mechanism on the 9-layer baseline with the original post-training quantization and follow a two-stage selection: first, a single-seed sweep over the mechanism’s key hyperparameter identifies a promising configuration; then, the single-seed winner is validated with 3 seeds to confirm the gain is not a noise artefact. In this way, improvements are attributable to the architecture change alone. The validated single-mechanism winners are subsequently composed hierarchically to obtain the final model and assess if their individual val-BpB loss gains are additive in nature.

Parallel Residuals replace the baseline’s sequential “attention then MLP” chain with a two-lane block. After the parallel-start layer psl, every block maintains two distinct residual states $\ell^{(0)}, \ell^{(1)}$: attention reads from $\ell^{(0)}$ and the MLP reads from $\ell^{(1)}$ (Fig. 1), so the two sublayers see different inputs rather than competing for the same residual subspace. Their outputs are then re-mixed into both lanes through a learned 2×2 routing matrix $\Lambda^{\text{post}} \in \mathbb{R}^{L \times 2 \times 2}$ scaled by per-sublayer residual gains $\Lambda^{\text{resid}} \in \mathbb{R}^{L \times 2}$, and the lanes are averaged at the end of the stack via $h = \frac{1}{2}(\ell^{(0)} + \ell^{(1)})$. The single hyperparameter psl controls how many blocks use this two-lane scheme; all details are in Sec. C.7.

PR Ablation We swept the parallel-start layer psl $\in \{0, 4, 7, -1\}$ (one seed each), where psl = -1 denotes the sequential-block control. The single-seed optimum was psl = 4, i.e. “entire decoder parallel”. A 3-seed validation at psl = 4 gave a -0.0044 BPB improvement over the baseline corresponding to 3.4σ of baseline std. We also verified that symmetric and asymmetric routing initialisations of Λ^{post} give a statistically indistinguishable result ($\Delta = 0.0002$).

Depth Recurrence is a parameter-free mechanism that re-runs a contiguous subset $\mathcal{R} \subseteq \{0, \dots, L-1\}$ of Transformer blocks for one extra pass with shared weights, positioned around the *encoder–decoder hinge* of the U-Net baseline. DR adds zero parameters “cost-free” on memory, but doubles FLOPs on every layer in $\mathcal{R}$. We accept this because our budget is fixed in *iterations*, not FLOPs: at matched step count every model receives the same number of gradient updates regardless of per-step compute. Under a FLOP-bounded setting a broader study should ablate the FLOP/BPB trade-off explicitly.

DR Ablation We swept the recurrent layer set $\mathcal{R} \in \{\{4\}, [4, 5], [3, 4], [5, 6]\}$ at the “hinge” between encoder and decoder, finding the inner-hinge configuration $\mathcal{R} = [3, 4]$ as the single-seed optimum. A follow-up sweep on the horizon ($[3, 4, 5]$, $[2, 3, 4, 5]$, $[3, 4, 5, 6]$, etc.) identified $\mathcal{R} = [2, 3, 4, 5]$ as the strongest configuration. A 3-seed validation gave 1.3022 ± 0.0029 BPB, a 0.0079 BPB improvement over the baseline corresponding to 6.1σ of baseline noise.

Attention Output Gate is a per-head, per-token soft that learns to amplify or reduce the contribution of each attention head. After the multi-head attention produces $y \in \mathbb{R}^{B \times T \times H \times d}$, the gate computes

$$g = 2 \sigma(W_g x_{:, :, 1:w_g}) \in \mathbb{R}^{B \times T \times H}$$

where $W_g \in \mathbb{R}^{H \times w_g}$ is a learned linear map from a narrow w_g -channel slice of the block’s input to one scalar per head, and σ denotes the sigmoid function. The gated attention output is thus $y_{b,t,h,:} \cdot g_{b,t,h}$, i.e. each head is rescaled by its own per-token scalar dial before the output projection W_O to allow for additional flexibility in rescaling important channels.

Attn Gate Ablation We swept the gate input width $w_g \in \{6, 8, 10, 12, 24\}$ and the source of the gate input (pre-projection query features vs. post-projection attention features). The query-feature source gave a slight regression vs. the disabled control while the standard post-projection source was uniformly beneficial. A 3-seed validation at $w_g = 12$, src = proj, gave 1.3058 ± 0.0008 BPB, a 0.0043 BPB improvement over the baseline corresponding to 3.4σ of baseline noise. The narrower $w_g = 8$ variant gave a marginally better mean (-0.0059 vs. baseline); we report $w_g = 12$ as the default because it composes slightly better with PR and DR.

Attention Residuals We created and evaluated 3 attention-residual variants to improve signal routing/propagation. The *Full Attention Residual* variant replaces the residual mixing and U-net skip aggregation with a depth-wise attention mixture over previous block states, but in this setting it degraded performance by 0.0180 BPB compared to baseline. The *Block Attention Residual* variant, which groups depth states into coarse block summaries before applying attention, performed worst, reaching 1.4619 ± 0.0500 BPB, suggesting that the current block implementation was unstable or too complex for this small GPT configuration. In contrast, the simpler *Skip-Attention* variant, which only replaces the fixed encoder–decoder skip bridge with a softmax-weighted aggregation over stored skip states and the current decoder state, was the only successful modification, improving the mean final BPB to 1.3064 ± 0.0011 (-0.0037 BPB relative to baseline). All details are in Appendix C.12.

Composition at $L = 9$: The first 3 mechanisms, PR, DR and attention gates, compose near-linearly: the full PR+DR+Gate stack at $L = 9$ reaches 1.2927 ± 0.0007 BPB (-0.0174 vs. baseline), matching the sum of single-mechanism gains exactly (100% composition efficiency). The pairwise PR×DR composition reaches 93% efficiency (-0.0114 vs. predicted -0.0123). This near-additivity is evidence that PR, DR, and Gate operate on different axes: routing, depth-iterated capacity, and per-head attention modulation. Differently, the *Skip Attention* variant of Attention Residuals, when combined with the 3 previous mechanisms, led to a steep increase in validation BpB to about 1.35745 and so it was not included in the final model.

Discarded Architecture Choices Following paper Zhai [2026] and taking inspiration from several submissions to the original *Parameter-Golf Challenge*, we also implemented Exclusive Self-Attention (XSA), but discarded it for the final model due to its insignificant contribution to val-BpB. All details are in appendix C.11.

4 Quantization Method and Experiments

Given the 16 MB artifact budget and given that the quantised baseline is already 15.78 MB, saving other space is essential: indeed any parameter-heavy architectural addition becomes feasible only if it is used a stronger quantization. We ablated the baseline’s quantization choices in 3 directions (Appendix C.20): (i) **bitwidth**: we ablated int4, int6 and int8 bitwidths, (ii) **layer sensitivity**: We tested the impact on *val_bpb* of quantizing the embeddings, the first transformer block, the last transformer block, the attention scores of all transformer blocks and the MLP parameters of all transformers blocks, (iii) **per-channel vs. per-tensor scales**: we tested if scaling over the entire tensor kept the same *val_bpb* compared to scaling over channels. Lastly, we tried to implement the post-training quantization techniques GPTQ, AWQ, NF4 and one of the most common approaches used in the leaderboard: GPTQ+LQER. The main results of the ablations will be later discussed in section 5.3

4.1 Final quantization setup

Based on the results of the previously described ablations, we adopt for the final model a *mixed-bitwidth* scheme: non-recurrent matrices are quantised at INT6, recurrent layers and the embedding at INT8, while sensitive control vectors (attention/MLP scales, residual mixers) are kept in fp16/fp32. The model is trained with quantization-aware training (QAT) from 25% of training, and the artifact is serialised with byte-shuffled brotli compression. This quantization approach will be justified in section 5.3

5 Results

5.1 Combined Architecture and Quantization at $L = 11$

We combine the validated architecture mechanisms with the mixed-bitwidth quantization scheme and increasing depth to $L = 11$ to use the additional memory budget, we re-ablate the two depth-dependent hyperparameters (parallel-start layer and recurrent layer set). Both re-ablations confirm small shifts: $\text{psl} = 4$ remains optimal (multi-seed 1.2812 ± 0.0007 vs. 1.2843 ± 0.0026 at $\text{psl} = 8$, where the single-seed sweep had favoured $\text{psl} = 8$ as a -1.2σ outlier), and $\mathcal{R} = [3, 4, 5]$ is preferred over the $L = 9$ winner $[2, 3, 4, 5]$ because of much smaller seed variance ($3.7\times$) and artifact size of 0.46 MB. The Gate hyperparameter $w_g = 12$ instead carries over unchanged.

The final stack reaches 1.2812 ± 0.0007 BPB at 15.36 MB (5 seeds) amounting to a reduction by 0.0289 (2.2%) in BpB over the OpenAI baseline at slightly smaller artifact size, with a post-quantization BPB increase of only $+0.004$ (comparable to the baseline’s $+0.0036$).

5.2 Why do the architectural additions help?

The clearest interpretive signal comes from *parallel residuals* (PR). The learned 2×2 routing matrix in each parallel layer converges to a strongly asymmetric pattern: attention writes almost exclusively to one lane and the MLP to the other, and the pattern is consistent across seeds (App. C.10, Fig. 29). We read this as the two lanes giving attention and MLP non-overlapping write channels into the residual stream, removing the implicit competition for the same subspace that the sequential block forces.

Crucially this specialisation is *emergent*: pre-initialising the routing to the asymptotic lane-diagonal solution *hurts* final loss under matched compute (Fig. 31), so the symmetry-breaking trajectory matters, not just the endpoint. The λ_{resid} scalars stay bounded throughout training (Fig. 30), confirming neither lane is shut off.

For *depth recurrence* (DR) and the *attention gate* (AG) the evidence is mechanistically shallower but consistent with non-trivial roles. DR’s gain is concentrated in the mid-stack and plateaus quickly with the number of recurrent passes (App. C.10), which points to it adding *effective depth* via shared computation rather than raw capacity. AG exhibits a clear interior optimum in gate width (App. C.10): both narrower and wider gates underperform, ruling out a pure parameter-count explanation.

5.3 Justification of final quantization and results

We arrive at the final quantization setup by combining the ablations of Sec. 4. The bitwidth ablation (Fig. 6) shows int6 saves 3.82 MB over int8 at $+0.0575 \text{ val_bpb}$, and the layer-sensitivity analysis (Fig. 12) shows that keeping embeddings unquantized is strongly beneficial; we therefore use int6 throughout except for embeddings and recurrent layers (int8, to prevent quantization error being amplified through the recurrence). Moreover, sensitive vectors (attention/MLP scales, residual mixers) are not quantized. Symmetric per-channel scaling is found to be the best scaling strategy (Fig. 16). Quantization Aware Training is not strictly essential at this scale but is retained as a hedge for deeper future variants. Advanced PTQ (GPTQ, AWQ, NF4) only helps meaningfully at int4 in our setting, which is itself uncompetitive (Figs. 10, 18, 9; Tabs. 3, 4). The resulting pre-/post-quant gap is 0.004, well below the 0.0084 of the three best Parameter Golf submissions and close to the baseline’s 0.0036 (Tab. 5).

Configuration	BPB ↓	$\Delta\text{BPB}_{\text{quant}}$	$\Delta\text{BPB}_{\text{base}}$	Size (MB)
Baseline GPT	1.3101 ± 0.0013	0.0036 ± 0.0002	—	15.78
<i>Single-mechanism ablations at $L = 9$</i>				
+ Parallel Residuals	1.3057 ± 0.0016	0.0035 ± 0.0001	-0.0044	15.82
+ Depth Recurrence	1.3022 ± 0.0029	0.0037 ± 0.0000	-0.0079	15.75
+ Attention Gate	1.3058 ± 0.0008	0.0039 ± 0.0001	-0.0043	15.77
<i>Pairwise and full compositions at $L = 9$</i>				
PR $\times$ DR	1.2987 ± 0.0028	0.0033 ± 0.0003	-0.0114	15.80
PR $\times$ DR $\times$ Gate	1.2927 ± 0.0007	0.0032 ± 0.0001	-0.0174	15.79
<i>Final GPT model</i>				
PR + DR + Gate + Mixed Quant + QAT	1.2812 ± 0.0007	0.0040 ± 0.0002	-0.0289	15.36

Table 1: Architecture ablations at $L=9$ (3 seeds) and the final $L=11$ model (5 seeds). $\Delta\text{BPB}_{\text{base}}$ is relative to the OpenAI baseline and $\Delta\text{BPB}_{\text{quant}}$ is the difference between the pre- and post-quantization model

6 Conclusions

We combined three low-parameter-cost architectural mechanisms (Parallel Residuals, Depth Recurrence, Attention Output Gate) with a mixed-bitwidth INT6/INT8 scheme trained under QAT. The approach to the project is chiefly mechanistic: each addition was introduced in isolation, evaluated against the sequential baseline at matched compute, and kept only when its ablation revealed an interpretable role rather than a pure parameter-count gain, i.e. emergent lane specialisation for parallel residuals and mid-stack effective depth for depth recurrence.

Two limitations follow directly from this methodology. First, most single-axis ablations were run at one seed under a fixed 5,000-step budget, with multi-seed validation reserved for the components we kept, so second-order interactions are only partially characterised. Second, our experiments see $\sim 6.6 \times 10^8$ training tokens against $\sim 1.05 \times 10^{10}$ for a nominal $8 \times \text{H100}$, 10-minute leaderboard run – $\sim 16 \times$ fewer tokens ($\sim 94\%$ smaller), the product of a $4 \times$ smaller per-step batch and a $4 \times$ shorter step budget (see Sec. C.4). The natural next step is to apply the same recipe at full compute, both to test whether the mechanistic findings still hold or if Attention Residuals prevail over the baseline’s fixed residual routing structure for longer training.

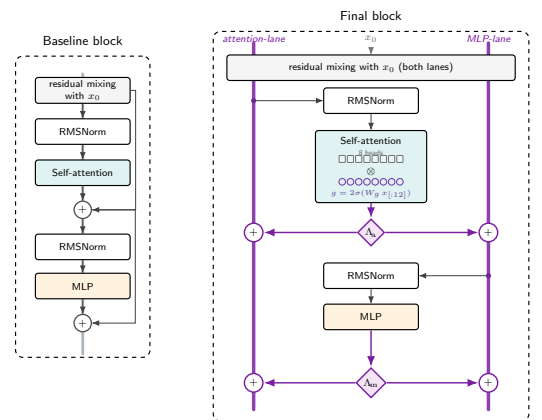
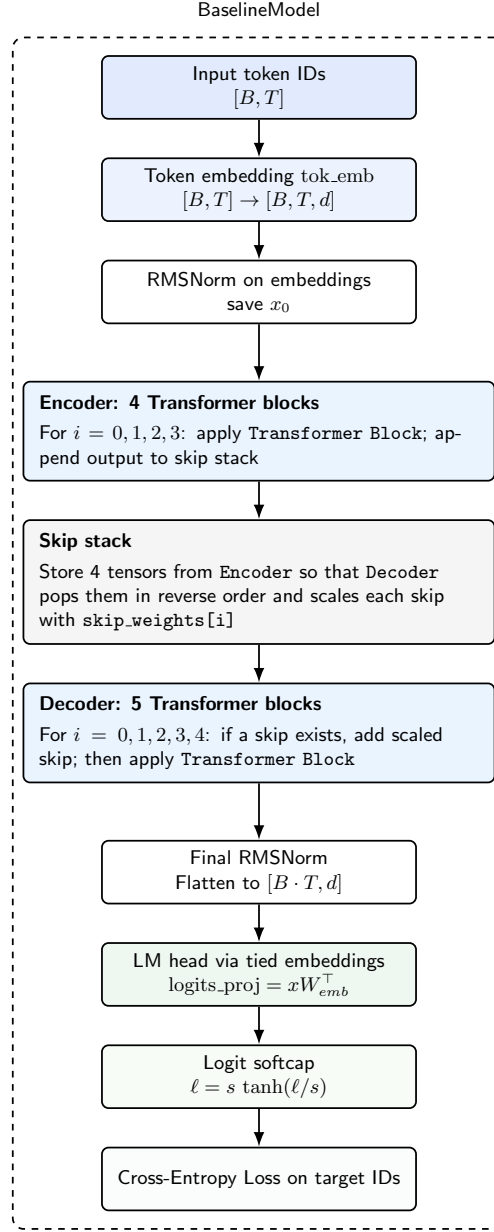


Figure 1: Comparison of baseline transformer block and our final block. The two-lane parallel residual and attention output gate are visible here; depth recurrence operates at the stack level, Figure 3.

References

- [1] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- [2] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.
- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. URL <https://arxiv.org/abs/2305.14314>.
- [4] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2023. URL <https://arxiv.org/abs/2210.17323>.
- [5] harborglowvintage oss. SP8192 + Depth Recurrence + Parallel Residuals + TTT + SDCLIP + GPTQ-Brotli. Pull Request #1974, openai/parameter-golf repository, 2026. URL <https://github.com/openai/parameter-golf/pull/1974>.
- [6] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference, 2017. URL <https://arxiv.org/abs/1712.05877>.
- [7] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024. Blog post.
- [8] Team Kimi, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, Yutian Chen, Junjie Yan, Ming Wei, Y. Zhang, Fanqing Meng, Chao Hong, Xiaotong Xie, Shaowei Liu, Enzhe Lu, Yunpeng Tai, Yanru Chen, Xin Men, Haiqing Guo, Y. Charles, Haoyu Lu, Lin Sui, Jinguo Zhu, Zaida Zhou, Weiran He, Weixiao Huang, Xinran Xu, Yuzhi Wang, Guokun Lai, Yulun Du, Yuxin Wu, Zhilin Yang, and Xinyu Zhou. Attention residuals, 2026. URL <https://arxiv.org/abs/2603.15031>.
- [9] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- [10] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations (ICLR)*, 2020.
- [11] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration, 2026. URL <https://arxiv.org/abs/2306.00978>.
- [12] MarioPaerle. SmearGate + Attention Output Gate + Legal TTT, April 2026. URL <https://github.com/openai/parameter-golf/pull/1667>. Record submission to openai/parameter-golf; reported val_bpb=1.07139.
- [13] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free, 2025. URL <https://arxiv.org/abs/2505.06708>.
- [14] Shuangfei Zhai. Exclusive self attention, 2026. URL <https://arxiv.org/abs/2603.09078>.
- [15] Cheng Zhang, Jianyi Cheng, George A. Constantinides, and Yiren Zhao. LQER: Low-rank quantization error reconstruction for LLMs. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL <https://arxiv.org/abs/2402.02446>.

A General Diagrams for Baseline vs Final Model Comparison



B = batch size, T = sequence length, d = model dimension.

Figure 2: Diagram of Baseline Model implemented by OpenAI for Parameter-Golf Challenge

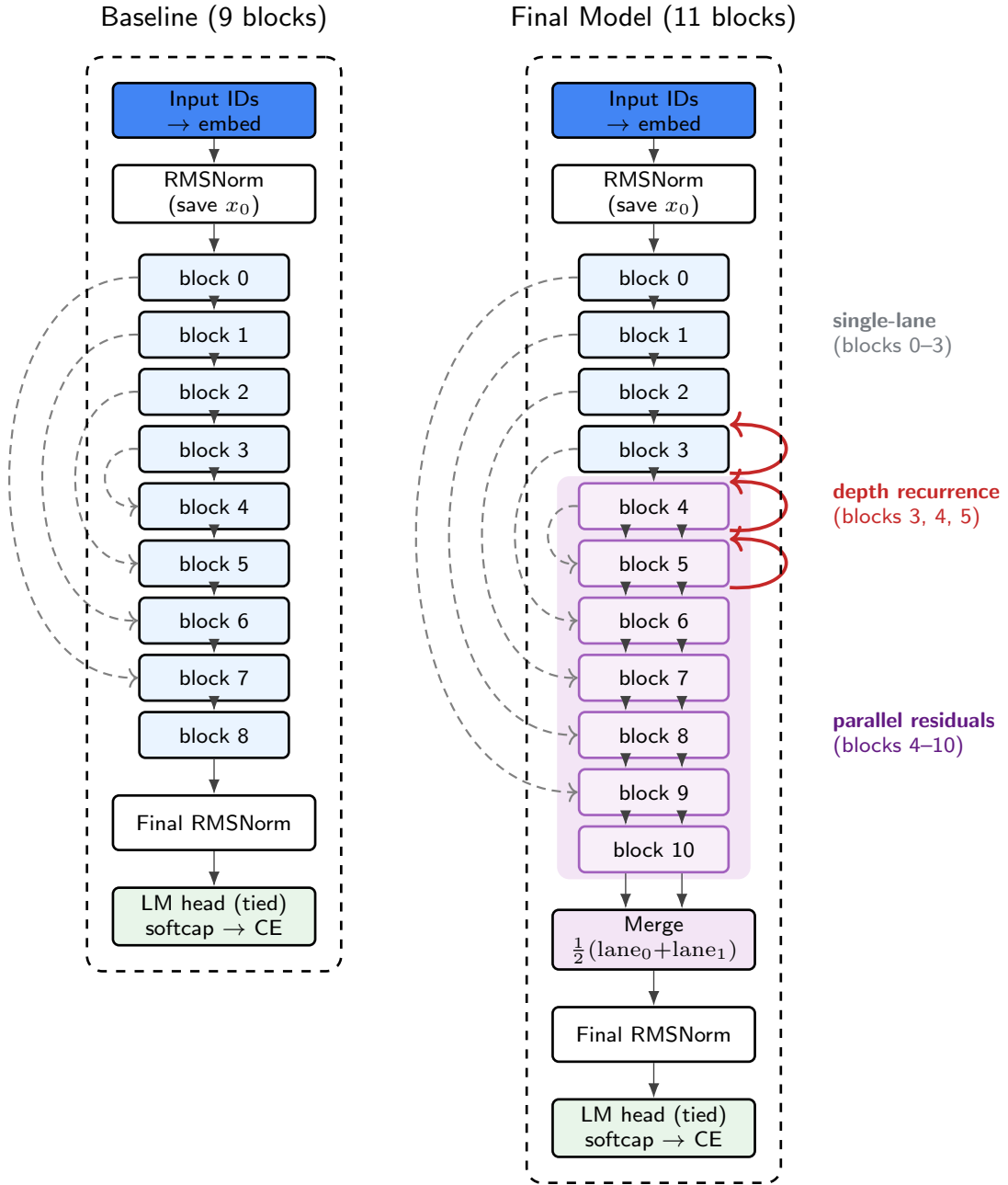


Figure 3: Diagram of final model with Parallel residuals, Attention Gates and Depth Recurrence compared with Baseline Model

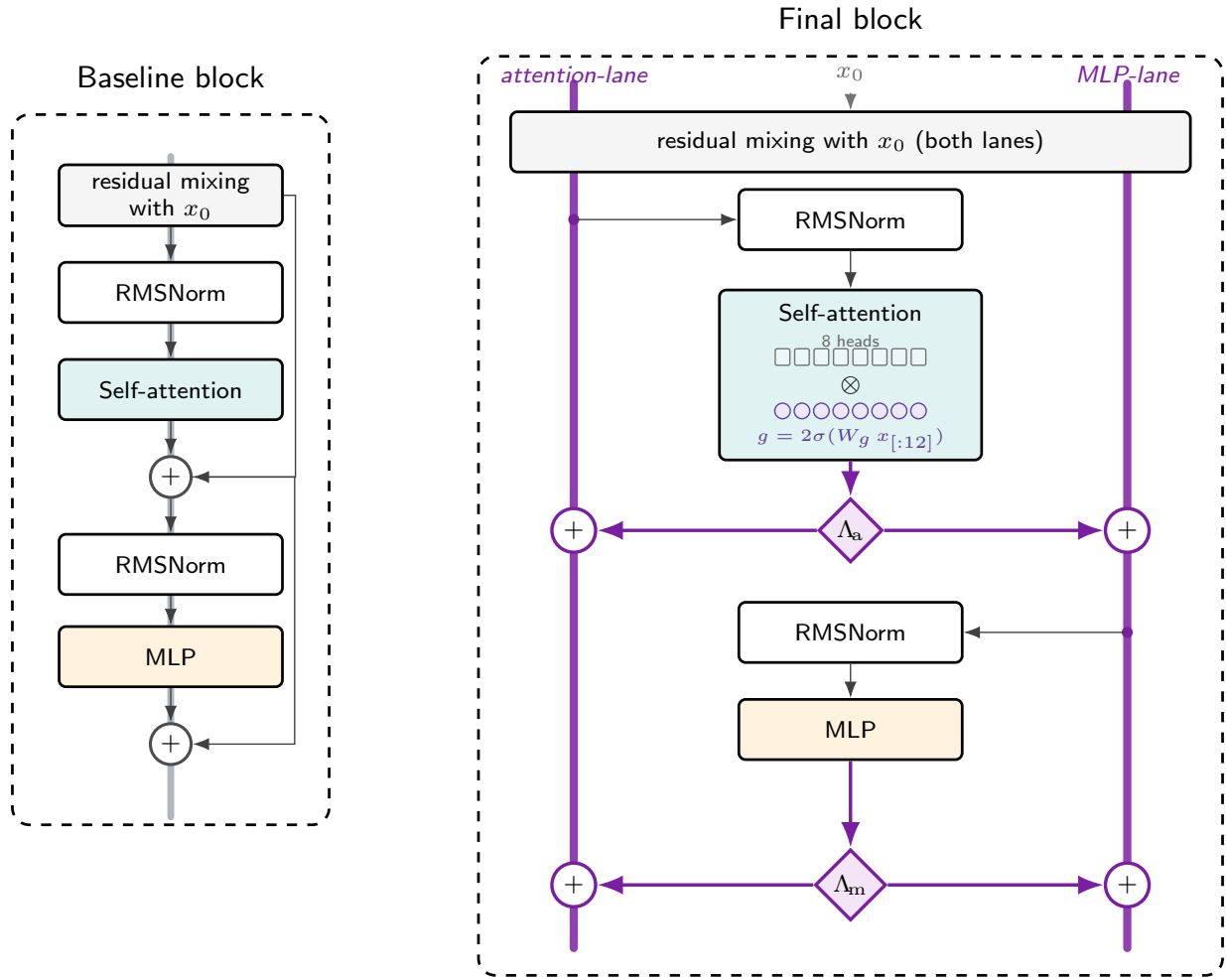


Figure 4: Diagram of final Transformer Block with Parallel residuals, Attention Gates and Depth Recurrence compared with Baseline Model

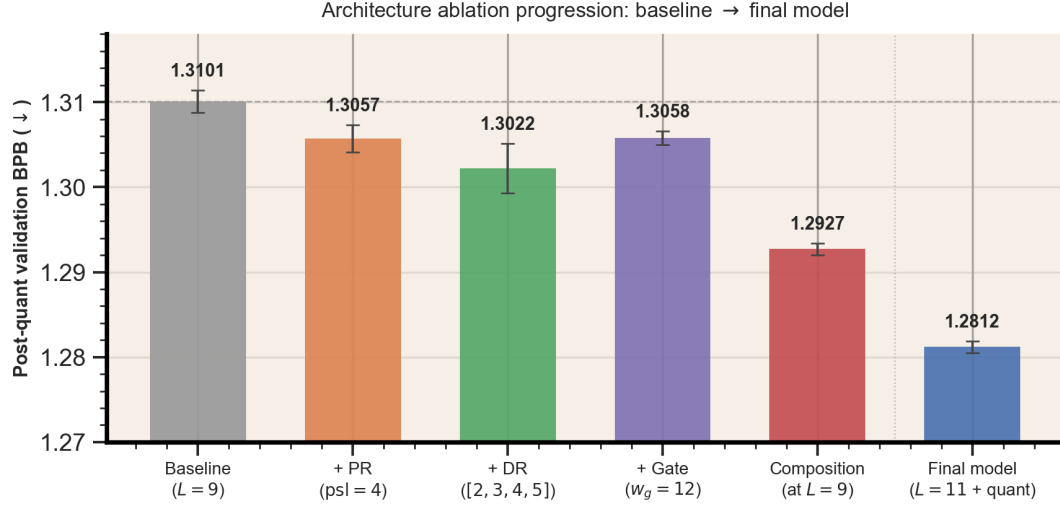


Figure 5: Architecture ablation progression. The full PR + DR + Gate composition at $L = 9$ reaches 1.2927 BPB (-0.0174 vs. baseline, 100% linear composition efficiency); the final $L = 11$ stack with mixed INT6/INT8 quantization reaches 1.2812 BPB (-0.0289 vs. baseline) at slightly smaller compressed size. Error bars: ± 1 seed std.

B Quantization Experiments' Graphs and Tables

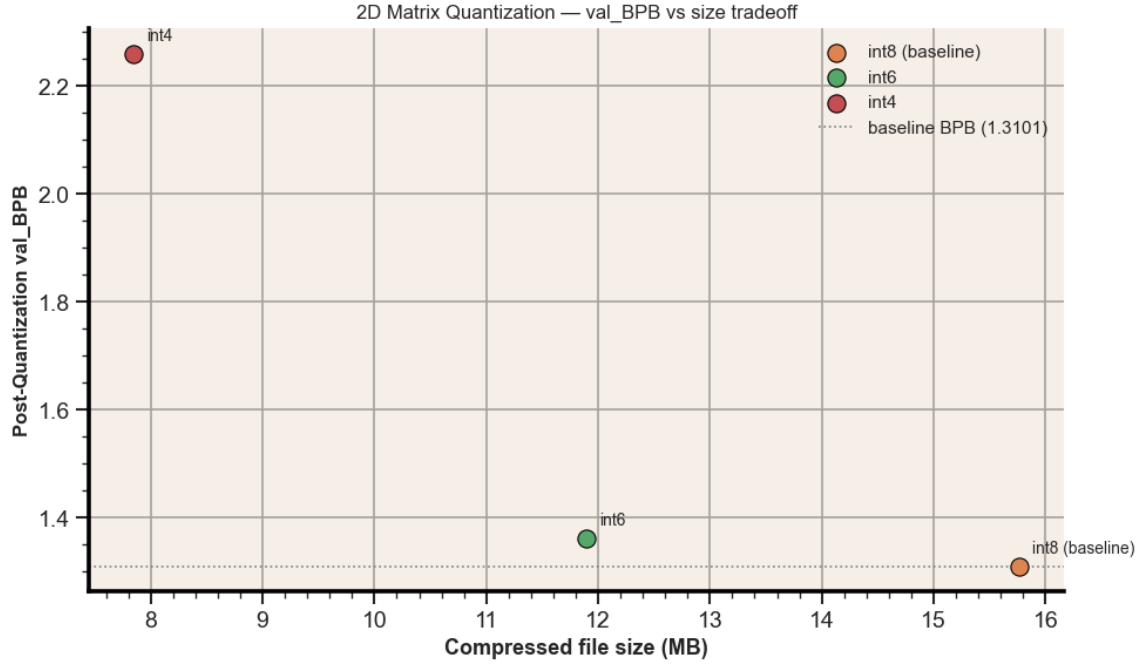


Figure 6: Ablation of the change of bidwidth in the baseline model quantization. On the x axis there is the dimension of the compressed and quantized model, while on the y axis the post quantization val bpb.

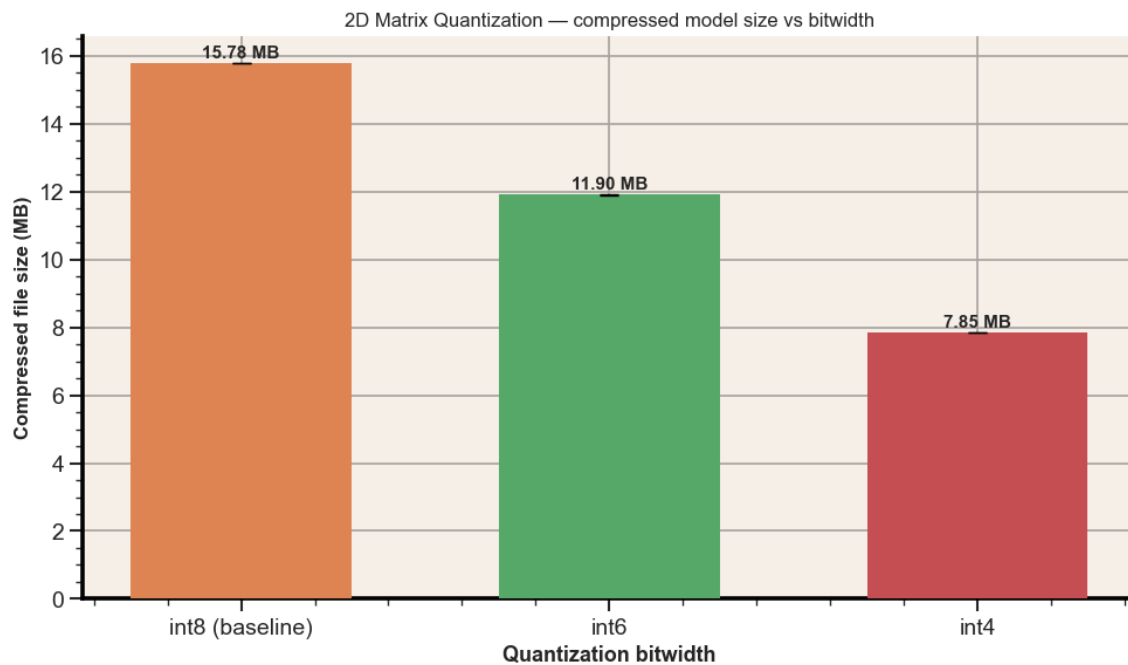


Figure 7: Dimension of the baseline model vs bitwidth used to quantize.

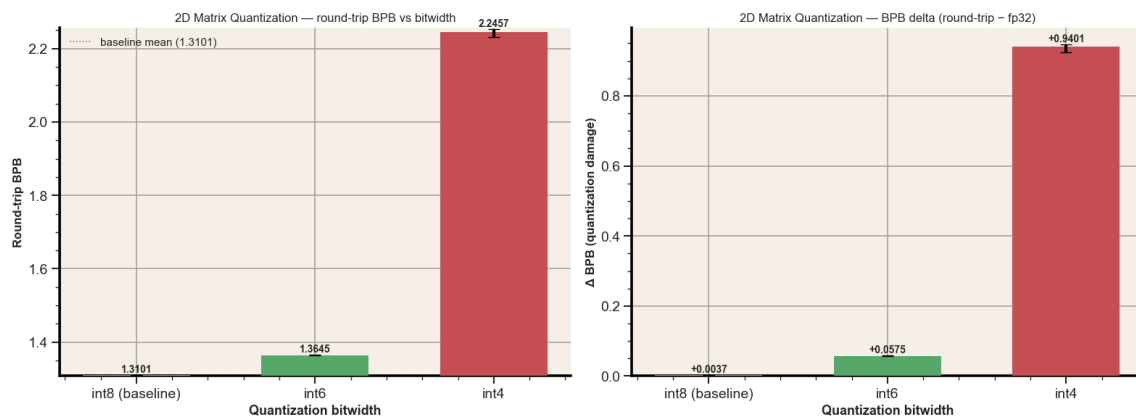


Figure 8: *val_bpb* of baseline model vs bitwidth used to quantize.

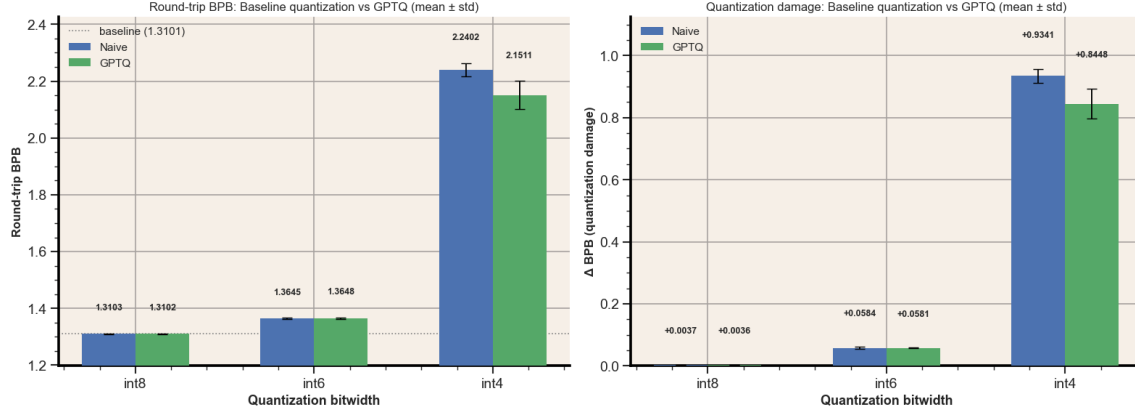


Figure 9: Comparison of post-quantization *val_bpb* between the baseline quantization and GPTQ

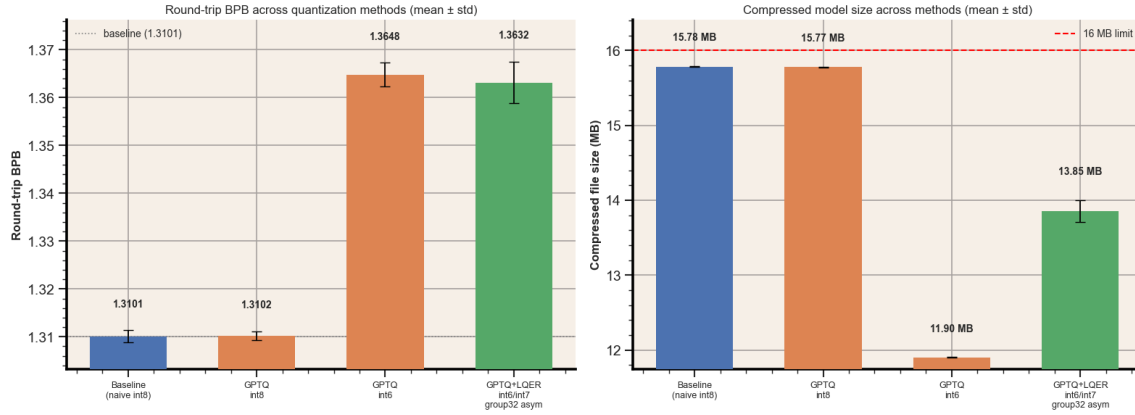


Figure 10: Differences in *val_bpb* between the baseline quantization, GPTQ quantization with int8, GPTQ quantization with int6, GPTQ quantization (int6) with asymmetric LQER + int7 quantization of embeddings

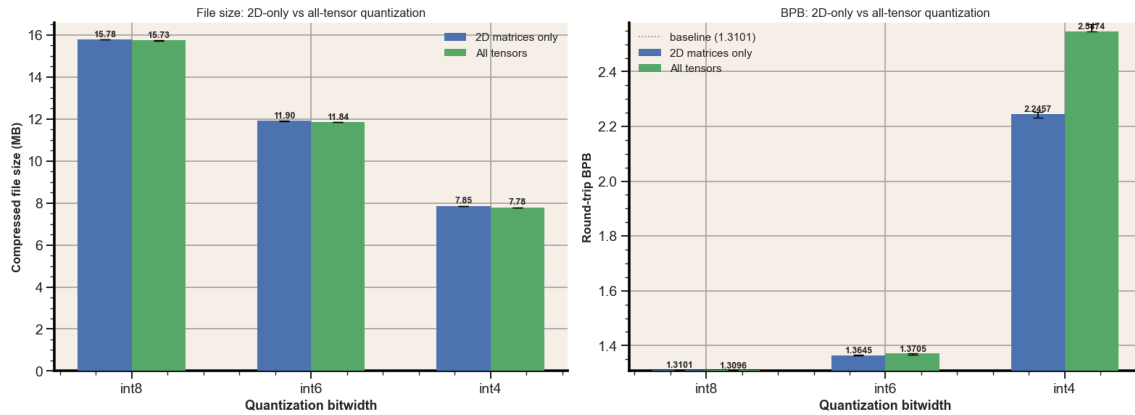


Figure 11: Differences in *val_bpb* by quantizing all the tensors vs not quantizing sensitive ones

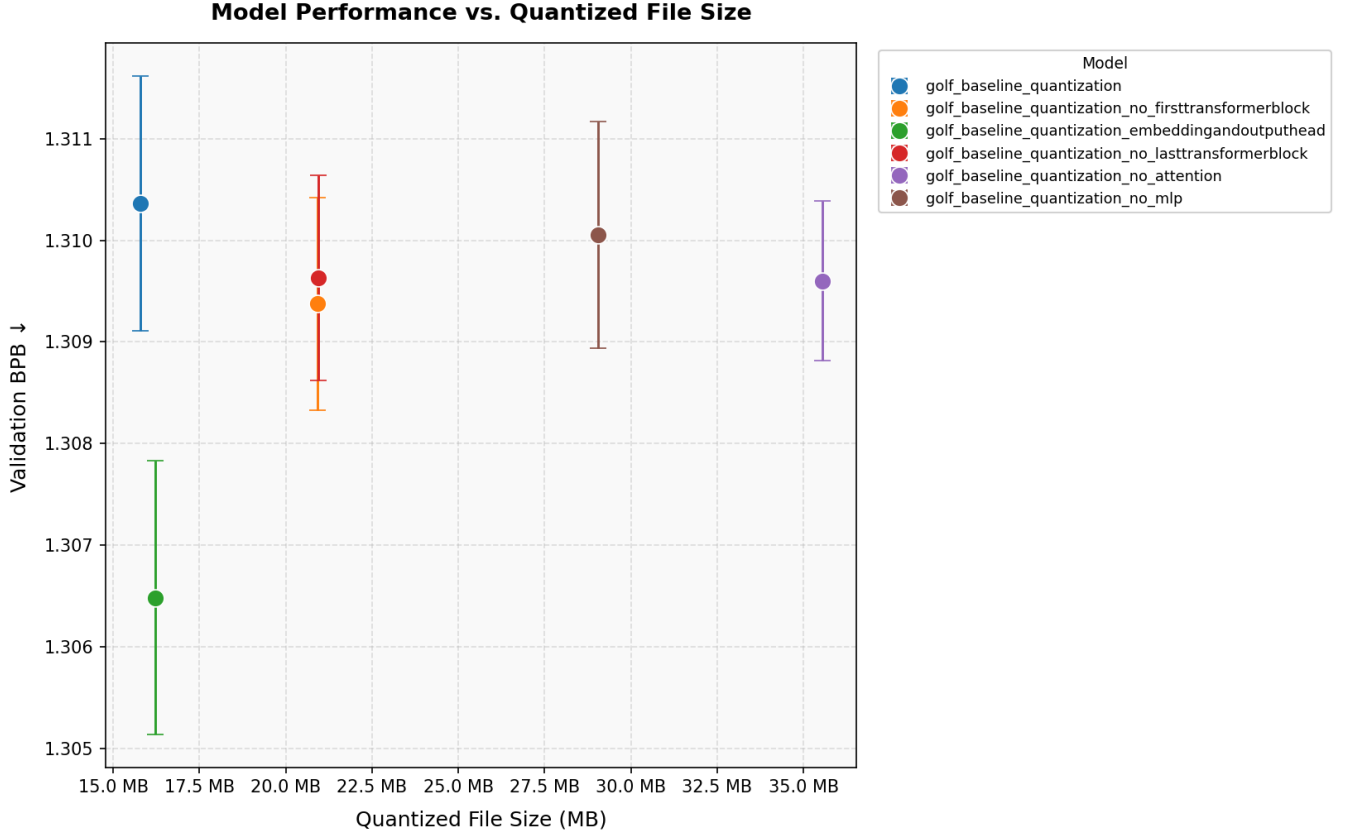


Figure 12: We compare the validation and file size between differently quantized models. All of the model utilise the quantization style of the baseline (transforming the weights from FP32 to INT8), but we vary which parts of the model are quantized. *Blue*: Baseline model, quantized most of the model except for *Green*: the token embeddings and output head *Yellow*: the first transformer block, *Red*: the last transformer block, *Purple*: the attention for each transformer block, *Brown*: the mlp for each transformer block.

	bits	granularity	symmetry	calibration	n_seeds	final_val_bpb_mean	final_val_bpb_std	delta_bpb_mean	quant_file_MB_mean
0	8	per_channel	symmetric	uncalibrated	3	1.310128	0.001125	0.003847	15.751136
1	8	per_channel	symmetric	percentile	3	1.310513	0.001189	0.003996	15.750205
2	8	per_tensor	symmetric	percentile	3	1.326211	0.005476	0.019676	14.023046
3	8	per_tensor	symmetric	uncalibrated	3	1.329274	0.003220	0.022915	13.803767
4	4	per_channel	symmetric	uncalibrated	3	2.574858	0.032795	1.268263	7.817289
5	4	per_channel	symmetric	percentile	3	2.613341	0.033294	1.306778	7.816488
6	8	per_tensor	asymmetric	uncalibrated	3	4.105208	0.000000	2.798569	14.070386
7	8	per_tensor	asymmetric	percentile	3	4.105208	0.000000	2.798795	14.167489
8	4	per_channel	asymmetric	percentile	3	4.287840	0.301076	2.981193	8.346790
9	4	per_channel	asymmetric	uncalibrated	3	4.509013	0.557465	3.202720	8.347866
10	4	per_tensor	symmetric	percentile	3	6.576486	0.377780	5.270114	6.001352
11	4	per_tensor	symmetric	uncalibrated	3	6.604472	0.373413	5.298057	5.782419
12	4	per_tensor	asymmetric	uncalibrated	3	6.915064	1.280510	5.608354	6.190904
13	8	per_channel	asymmetric	uncalibrated	3	7.527228	3.148215	6.221132	16.046714
14	4	per_tensor	asymmetric	percentile	3	8.045439	1.868324	6.738971	6.259384
15	8	per_channel	asymmetric	percentile	3	15.346187	9.756486	14.039690	16.045366

Table 2: Full quantization sweep results.

Method	Granularity	Calibration	Final BPB ↓	ΔBPB vs pre-quant	Size
INT8 uniform symmetric	per-channel	uncalibrated	1.3101 ± 0.0011	+0.0038	15.75 MB
INT4 uniform symmetric	per-channel	uncalibrated	2.5749 ± 0.0328	+1.2683	7.82 MB
NF4	per-channel	uncalibrated	1.8163 ± 0.0222	+0.5100	7.91 MB
NF4	per-channel	percentile-clipped	1.8211 ± 0.0208	+0.5148	7.91 MB
NF4	per-tensor	uncalibrated	7.5238 ± 2.5315	+6.2175	6.47 MB

Table 3: Basic quantization design study results. Lower BPB is better.

Ablation	Comparison	Effect on BPB	Interpretation
Granularity, INT4 symmetric raw	per-channel - per-tensor	-4.0296	per-channel is essential
Granularity, INT8 symmetric raw	per-channel - per-tensor	-0.0191	small but positive effect
Symmetry, 4-bit per-channel raw	asymmetric - symmetric	+1.9342	asymmetric worse
Symmetry, 8-bit per-channel raw	asymmetric - symmetric	+6.2171	asymmetric unstable
Calibration, 4-bit per-channel symmetric	percentile - raw	+0.0385	clipping slightly worse
NF4 granularity	per-channel - per-tensor	-5.7075	per-channel required
NF4 calibration	percentile - raw	+0.0048	negligible effect

Table 4: Ablation table for quantization design choices.

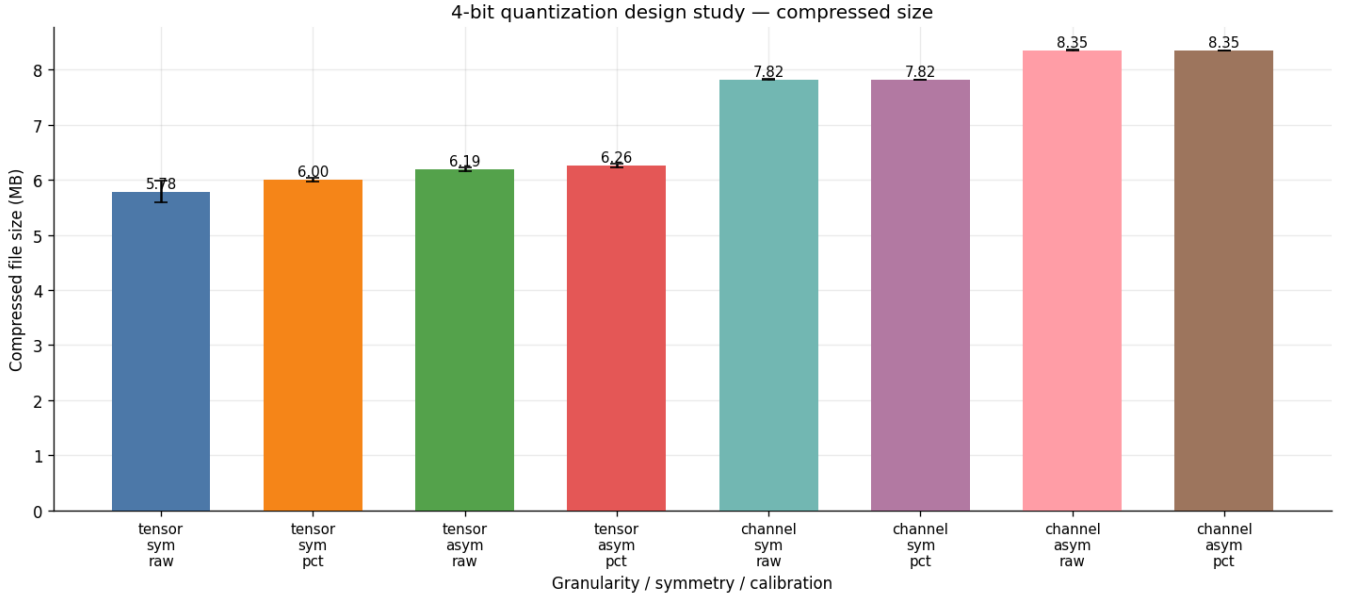


Figure 13: Compressed file size comparison across the 4-bit quantization configurations.

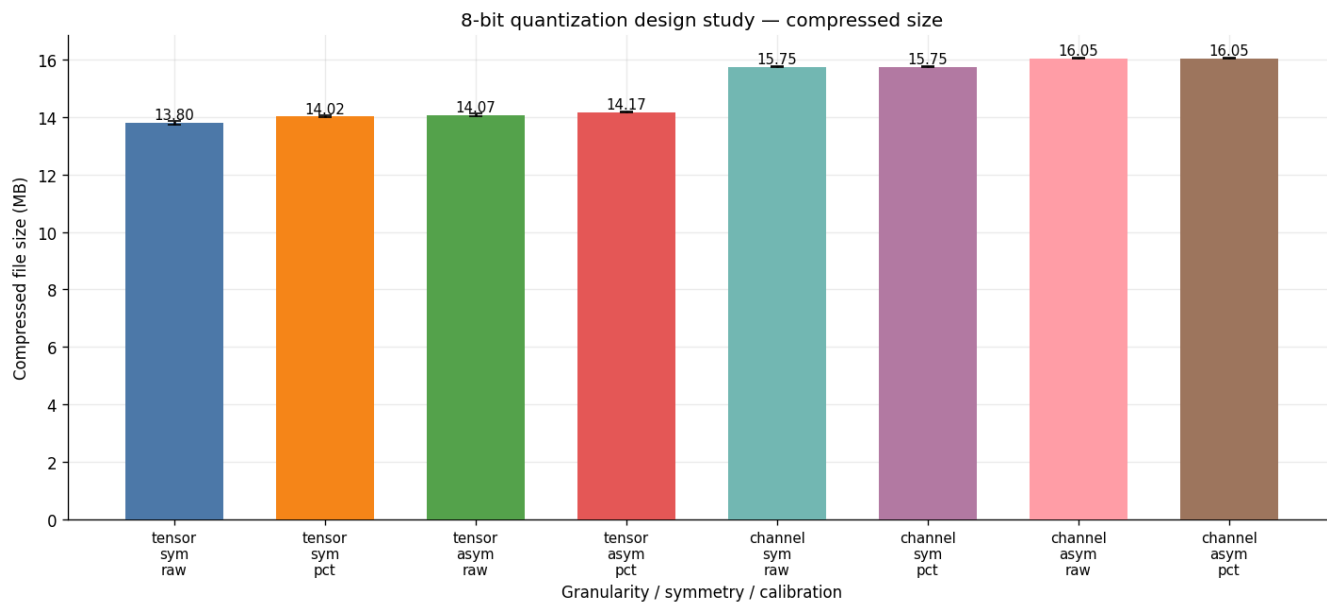


Figure 14: Compressed file size comparison across the 8-bit quantization configurations.

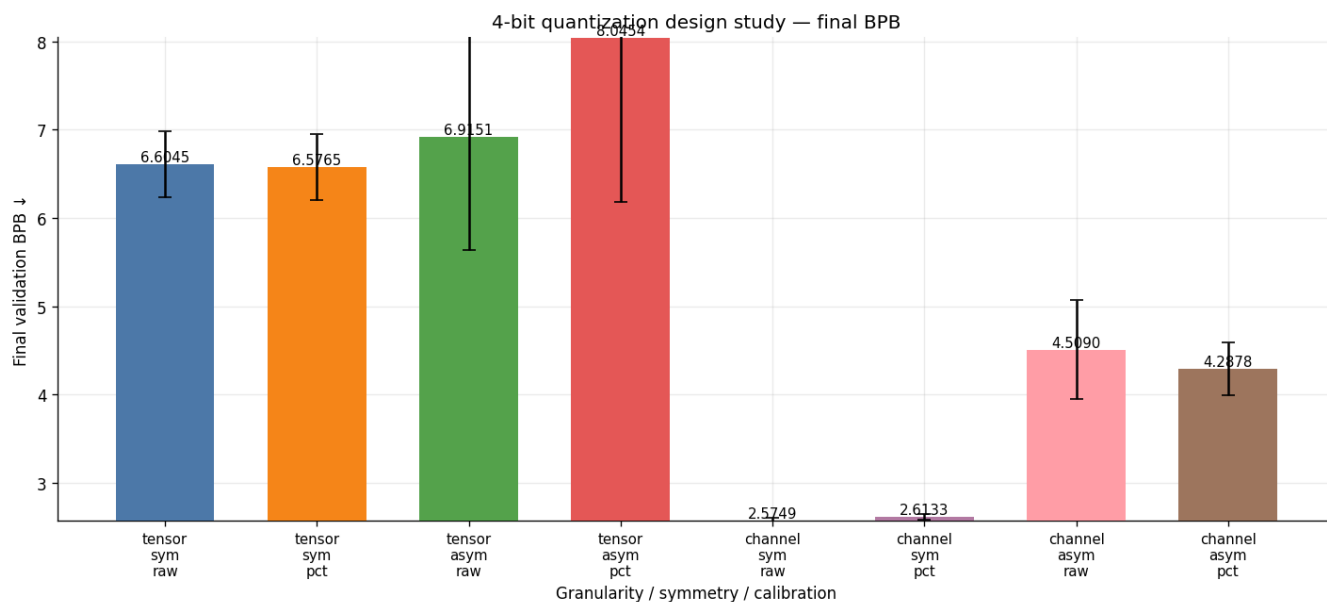


Figure 15: Final validation BPB comparison across the 4-bit quantization configurations.

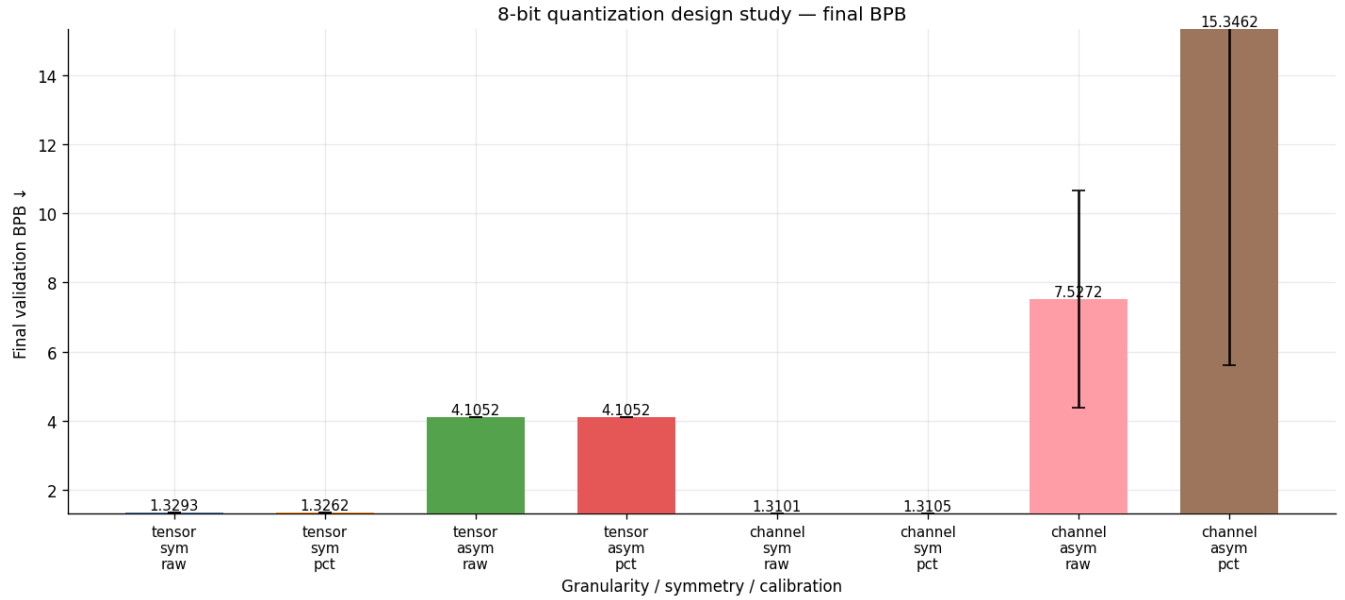


Figure 16: Final validation BPB comparison across the 8-bit quantization configurations.

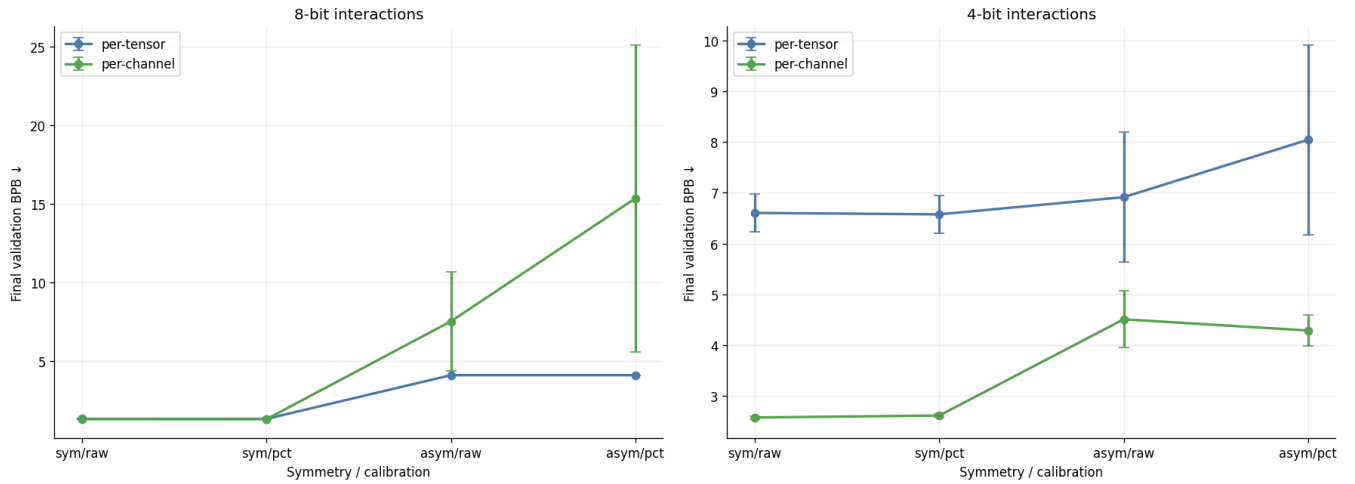


Figure 17: Interaction plots summarizing how granularity, symmetry, and calibration jointly vary across the quantization configurations.

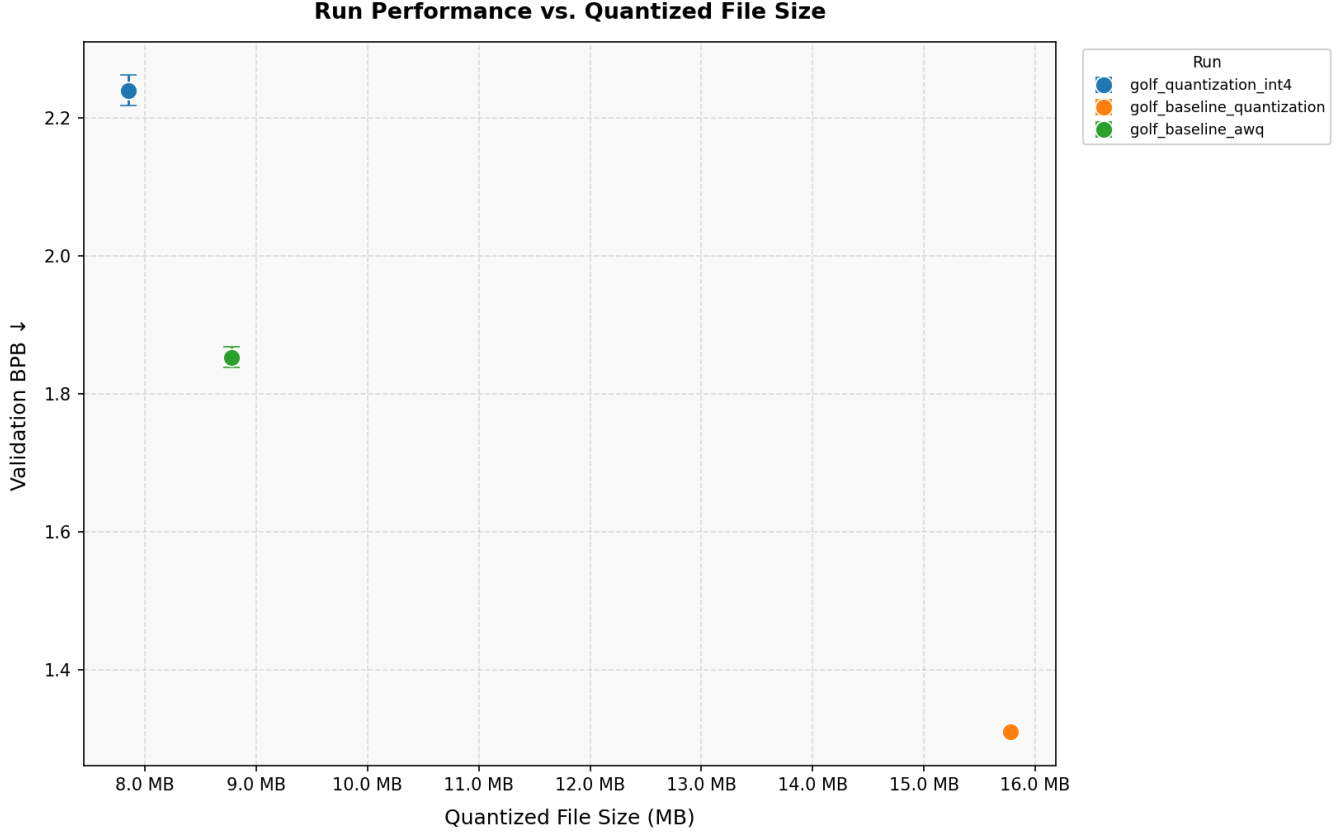


Figure 18: Compare AWQ (with salient weights at FP16 and the rest kept at INT4) to the baseline quantization and the baseline quantization with reduced weight dimension to INT4. The AWQ quantization achieved: `val_bpb` = 1.853514 and file size = 8779192.

Model	Pre-quant BPB	Post-quant BPB	Delta
Baseline	1.3065 ± 0.0010	1.3101 ± 0.0013	0.0036
Top submission #1	1.0607 ± 0.0004	1.0691 ± 0.0004	0.0084
Top submission #2	1.0601 ± 0.0005	1.0686 ± 0.0004	0.0084
Top submission #3	1.0619 ± 0.0002	1.0703 ± 0.0003	0.0084
Our model	1.2772 ± 0.0002	1.2811 ± 0.0004	0.0040

Table 5: Pre- and post-quantization validation BPB across the baseline, the top three challenge submissions, and our model. Values are mean $\pm$ standard deviation over 3 seeds. The delta is the BPB increase due to quantization (post – pre); a smaller delta indicates that the quantization pipeline better preserves the full-precision quality. The top three submissions of the challenge have the same delta because they used the same quantization technique.

C Technical Details of Baseline and Final Model Layers

C.1 Full overview of Baseline Model

Baseline Model Performance provided by OpenAI is a U-net-like GPT model that uses by default fixed hidden dimension size $d = 512$, sequence length $T = 1024$, and $L = 9$ Transformer blocks. Accordingly, the model has 17,059,912 trainable parameters and its compressed size, corresponding to 15.78 MegaBytes on average and mean compression ratio of 3.91, reaches almost the maximum memory allowed for the challenge. Its final validation BpB loss in our setting is then on average 1.3101 ± 0.0013 .

Baseline Architecture As shown in Figure 3, the model receives a sequence of token indices $x = (x_1, \dots, x_T)$, $x_i \in \{1, \dots, |V|\}$ which are mapped to vectors using an embedding matrix $E \in \mathbb{R}^{|V| \times d}$ whose resulting embedding sequence is then RMS-normalized to obtain

$h_0 = \text{RMSNorm}(Ex)$. Then, these embeddings are passed through the stack of $L = 9$ Transformer Blocks, which are arranged in a U-net autoencoder fashion: the first $L_{enc} = \lfloor L/2 \rfloor = 4$ blocks constitute the encoder part of the network while the remaining $L - L_{enc} = 5$ blocks serve as the decoder since they receive skip connections from the encoder blocks rescaled by learnable weight vectors $w_i \in \mathbb{R}^d$. This U-Net structure thus creates information highways across depth, allowing later transformer blocks to recover intermediate representations.

The Baseline’s *Transformer Blocks* consist of a pre-normalized causal self-attention layer followed by pre-normalized feed-forward MLP with expansion factor of 2 and ReLU^2 activation (details in appendix C.2). However, differently from traditional implementations, the block first computes a learnable elementwise mixture between current and initial residual stream by $\tilde{h} = m_0 \odot h_0 + m_1 \odot h$ where $m_0, m_1 \in \mathbb{R}^d$ are learnable vectors. This mechanism gives each block a direct pathway to the original token representation, while allowing to preserve standard residual behavior. Remarkably, also the residual stream of the outputs of the Attention and MLP layers are rescaled by 2 learnable vectors of length d allowing the layers to reduce or amplify particular channels’ signals with very low parameter cost.

After all Transformer blocks and a final RMS normalization $h_{\text{final}} = \text{RMSNorm}(h_L)$, logits are computed using the transpose of the token embedding matrix $o_t = h_{\text{final},t} E^\top$ and are passed through a soft-capping nonlinearity with hyper-parameter c given by $\hat{o}_t = c \tanh(o_t/c)$ to bound logits preventing extreme output scores. The model’s training loss is then computed by the standard next-token cross-entropy objective of GPT models.

Now, the model’s parameters can finally be optimized: matrix-valued Transformer parameters are updated with the help of the Muon optimizer, while embeddings and scalar or vector control parameters are optimized with Adam (further info in appendix C.5).

Baseline Quantization After training is completed, the model’s size is reduced by performing post-training symmetric int8 weight-only quantization. More concretely, here a conversion of large floating point tensors to int8 values in the range $[128, -128]$ is done with percentile clipping at 99.99984% in order to have more control over outliers. To do this we use per row scales for 2D weight matrices and per tensor scales for vectors and scalars. Crucially, small, non-float tensors and specific parameters like attention and MLP scales, residual mixing weights and query gains are kept in their original floating-point format (mostly float16 or float32), to preserve model’s learned pathways and context. On the other hand, quantized tensors, original dtypes and passthrough tensors are then serialized and compressed with zlib even if during evaluation the int8 weights are dequantized back to their respective higher precision rather than being used for int8 inference.

C.2 Transformer Block

For input h , the block first computes a learnable elementwise mixture between the current residual and the initial embedding stream:

$$\tilde{h} = m_0 \odot h_0 + m_1 \odot h$$

where $m_0, m_1 \in \mathbb{R}^d$ are learned vectors initialized with $m_1 = \mathbf{1}$, $m_0 = \mathbf{0}$ so that the block initially uses only the current residual stream and ignores the embedding stream, namely $m_1 = \mathbf{1}$, $m_0 = \mathbf{0}$. The attention sublayer is then applied in pre-norm form and its output is added back to the residual stream using a learned elementwise scale: (full description of $\text{Attn}(\cdot)$ is in C.3)

$$a = \text{Attn}(\text{RMSNorm}(\tilde{h})), \quad h' = \tilde{h} + s_{\text{attn}} \odot a$$

where $s_{\text{attn}} \in \mathbb{R}^d$ is initialized to $\mathbf{1}$.

And similarly for the MLP layer with a default expansion factor of 2:

$$l = \text{MLP}(\text{RMSNorm}(h')), \quad h_{\text{out}} = h' + s_{\text{mlp}} \odot l$$

where $\text{MLP}(u) = (\text{ReLU}(uW_1))^2 W_2 : W_1 \in \mathbb{R}^{d \times 2d}$, $W_2 \in \mathbb{R}^{2d \times d}$ and $s_{\text{mlp}} \in \mathbb{R}^d$ is also initialized to $\mathbf{1}$.

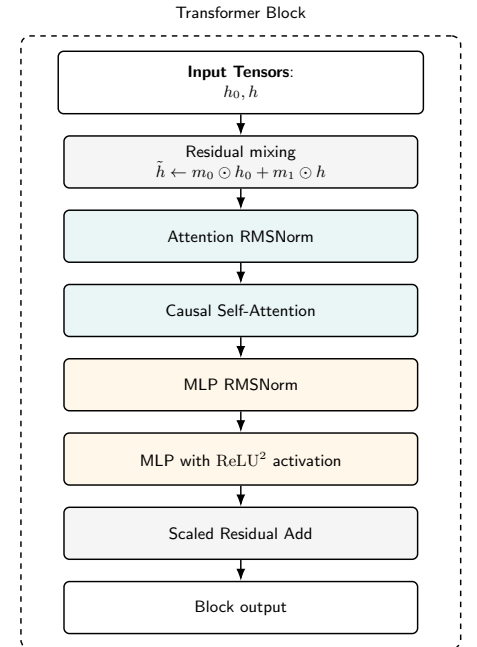


Figure 19: Diagram of Transformer Block in Baseline Model

C.3 Causal Self-Attention

The attention module uses grouped-query attention (GQA). Queries are projected into H heads, whereas keys and values are projected into only H_{kv} heads, with $H_{kv} < H$. For the default configuration, $H = 8$ and $H_{kv} = 4$. The head dimension is

$$d_h = d/H$$

For an input sequence $u \in \mathbb{R}^{B \times T \times d}$, the model computes

$$Q = uW_Q, \quad K = uW_K, \quad V = uW_V$$

where

$$W_Q \in \mathbb{R}^{d \times d}, \quad W_K, W_V \in \mathbb{R}^{d \times (H_{kv} d_h)}$$

Queries and keys are RMS-normalized at the head level before positional encoding is applied:

$$Q \leftarrow \text{RMSNorm}(Q), \quad K \leftarrow \text{RMSNorm}(K).$$

Rotary positional embeddings are then applied to Q and K . The queries are additionally scaled by a learned per-head gain parameter $g \in \mathbb{R}^H$, initialized to a constant value. The attention output is computed using causal scaled dot-product attention:

$$\text{Attn}(u) = \text{Concat} \left(\text{softmax} \left(\frac{QK^\top}{\sqrt{d_h}} + M_{\text{causal}} \right) V \right) W_O,$$

where M_{causal} masks future positions and $W_O \in \mathbb{R}^{d \times d}$ is the output projection. The output projection is zero-initialized, so that the attention branch initially contributes no perturbation to the residual stream.

C.4 Compute budget gap

All architecture ablations and the final $L=11$ model are trained on a single A100 for 5,000 optimisation steps at a per-step token budget of 131,072, totalling $5,000 \times 131,072 \approx 6.6 \times 10^8$ tokens seen during training. A nominal leaderboard run uses the baseline defaults of 20,000 steps at 524,288 tokens per step, calibrated to fit a ~ 10 min wall-clock budget on $8 \times H100$, totalling $\sim 1.05 \times 10^{10}$ tokens seen. Our experiments therefore expose the model to roughly 1/16 of the tokens of a full leaderboard run ($\sim 6\%$, i.e. $\sim 94\%$ smaller). This factor of 16 is the product of two independent $4 \times$ reductions — $4 \times$ fewer optimisation steps and a $4 \times$ smaller per-step batch — which compound multiplicatively in tokens-seen, and it sets the dominant axis along which our reported BPB differs from leaderboard top entries.

C.5 Optimizers

We use a split-by-parameter-type optimizer: matrix-valued Transformer weights are updated with Muon [7], while the token embedding matrix and all remaining scalar/vector control parameters (residual scales, residual-mixing coefficients, query gains, skip weights) are updated with Adam [9]. All optimizer hyperparameters are inherited unchanged from the simple baseline of `train_gpt.py`; only the training length (5000 iterations, warmdown over the last 750) and the per-step token budget were adapted to our compute setup.

Adam updates. For a parameter θ with gradient $g_t = \nabla_{\theta} \mathcal{L}_t$, Adam maintains exponential moving averages

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^{\odot 2},$$

the bias-corrected estimates

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

and applies

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

We use $(\beta_1, \beta_2) = (0.9, 0.95)$ and $\varepsilon = 10^{-8}$, with three separate Adam groups: the token-embedding group at learning rate $\eta_{\text{tied}} = 0.05$ (or $\eta_{\text{emb}} = 0.6$ when embeddings are untied), the scalar/vector control group at $\eta_{\text{scalar}} = 0.04$, and — only in the untied-embedding setting — the language-model head at $\eta_{\text{head}} = 8 \times 10^{-3}$.

Muon updates for matrix parameters. For each matrix-valued parameter $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, we first form a momentum-smoothed gradient

$$u_t = \mu_t u_{t-1} + g_t,$$

and a Nesterov-style gradient (the variant we use in all reported runs)

$$\tilde{g}_t = g_t + \mu_t u_t.$$

The Muon momentum μ_t is itself linearly ramped from $\mu_{\min} = 0.85$ to $\mu_{\max} = 0.95$ over the first $T_{\mu} = 500$ optimisation steps and held at $\mu_{\max}$ afterwards. Letting $G = \tilde{g}_t$ be the gradient matrix passed to the orthogonalisation step, we normalise

$$X_0 = \frac{G}{\|G\|_F + \varepsilon_{\text{NS}}}, \quad \varepsilon_{\text{NS}} = 10^{-7},$$

and apply the fifth-order Newton–Schulz iteration [?]]

$$X_{k+1} = a X_k + (b A_k + c A_k^2) X_k, \quad A_k = X_k X_k^\top,$$

for $k = 0, \dots, K-1$ with $K = 5$ and constants $a = 3.4445$, $b = -4.7750$, $c = 2.0315$. When G has more rows than columns ($d_{\text{out}} > d_{\text{in}}$), the iteration is applied to $G^\top$ and the result is transposed back, so that A_k is always formed on the smaller dimension — this lowers the per-step cost and improves the conditioning of the iterate. The orthogonalised update is

$$\Delta_t = \sqrt{\max\left(1, \frac{d_{\text{out}}}{d_{\text{in}}}\right)} X_K,$$

which restores the spectral scale lost by the input normalisation, and the parameter is updated with $\eta_{\text{matrix}} = 0.04$ via

$$\theta_{t+1} = \theta_t - \eta_{\text{matrix}} \Delta_t.$$

No gradient clipping is applied (`GRAD_CLIP_NORM` = 0).

Parameter grouping. Let Θ_{emb} denote the token-embedding parameters, Θ_{mat} the two-dimensional weight matrices of the Transformer blocks (Q/K/V/output projections and MLP weights), and Θ_{ctrl} the remaining scalar/vector and control tensors. The optimiser assignment is

$$\Theta_{\text{emb}} \rightarrow \text{Adam}(\eta_{\text{emb}}), \quad \Theta_{\text{mat}} \rightarrow \text{Muon}(\eta_{\text{matrix}}), \quad \Theta_{\text{ctrl}} \rightarrow \text{Adam}(\eta_{\text{scalar}}).$$

With tied embeddings (our default setting), the output classifier reuses the token-embedding matrix and is therefore optimised as part of Θ_{emb} with learning rate η_{tied} ; when embeddings are untied, the language-model head forms a separate Adam group with learning rate η_{head} .

Learning-rate schedule. A single multiplier $\rho(t)$ is shared across all three optimiser groups (*not* a per-group cosine):

$$\rho(t) = \begin{cases} 1, & 0 \leq t < T_{\text{total}} - T_{\text{wd}}, \\ \frac{T_{\text{total}} - t}{T_{\text{wd}}}, & T_{\text{total}} - T_{\text{wd}} \leq t < T_{\text{total}}, \end{cases}$$

i.e. the LR is held constant for most of training and linearly decayed to zero over the last T_{wd} steps. We use $T_{\text{total}} = 5000$ and $T_{\text{wd}} = 750$ for both the $L=9$ ablation runs and the final $L=11$ model. Each parameter group’s effective learning rate at step t is $\rho(t) \cdot \eta_{\text{group}}$.

Symbol	Description	Value
η_{matrix}	Muon LR (matrix weights)	0.04
η_{emb}	Adam LR (embeddings, untied)	0.6
η_{tied}	Adam LR (embeddings, tied – default)	0.05
η_{head}	Adam LR (untied LM head)	8×10^{-3}
η_{scalar}	Adam LR (scalar/vector params)	0.04
$\mu_{\text{min}} \rightarrow \mu_{\text{max}}$	Muon momentum (warmup over T_μ steps)	$0.85 \rightarrow 0.95$
T_μ	Muon momentum-warmup length	500
β_1, β_2	Adam EMA decays	0.9, 0.95
ε	Adam numerical floor	10^{-8}
ε_{NS}	Newton–Schulz normalisation floor	10^{-7}
K	Newton–Schulz iterations	5
(a, b, c)	Newton–Schulz coefficients	(3.4445, −4.7750, 2.0315)
T_{total}	Total training steps	5000
T_{wd}	Warmdown (linear ramp to 0)	750

Table 6: Optimizer hyperparameters used for all reported runs. All values match the simple baseline of `train_gpt.py`; only T_{total} , T_{wd} , and the per-step token budget differ from the original baseline.

C.6 Optimizer Hyperparameter Validation

Before adding architectural mechanisms, we verified that the baseline’s optimizer hyperparameters (Muon matrix LR, weight decay on Adam-managed and Muon-managed parameters) were already well-tuned for our setting ($L = 9$, 5000 iterations, single A100). All entries below are single seed (1337).

Matrix LR sweep. We swept the Muon learning rate for matrix parameters $\eta_{\text{matrix}} \in \{0.02, 0.04, 0.06, 0.08\}$ (Figure 20, left). The baseline default $\eta_{\text{matrix}} = 0.04$ is the single-seed optimum. Lower values under-train; higher values destabilise the late-training warmdown.

Weight decay grid. We tested all four combinations of weight decay on Adam-managed scalar parameters ($\lambda_{\text{Adam}} \in \{0, 0.04\}$) and Muon-managed matrix parameters ($\lambda_{\text{Muon}} \in \{0, 0.04\}$) (Figure 20, right). Any non-zero weight decay on Muon-managed parameters causes catastrophic collapse: BPB jumps from 1.31 to ~ 2.28 and the compressed artifact shrinks to < 1 MB, indicating that the matrix weights are crushed toward zero and lose representational capacity. We attribute this to Muon’s geometric (Stiefel-manifold-projected) update interacting poorly with the additive shrinkage of L2 regularisation. Scalar weight decay is not catastrophic but slightly hurts BPB (+0.0019). We therefore keep zero weight decay on both parameter groups throughout the rest of the project.

Implication. Together, the matrix-LR sweep and the weight-decay grid indicate that the baseline’s optimiser configuration is already at or near a local optimum for this compute setting: the default $\eta_{\text{matrix}} = 0.04$ wins the LR sweep and any non-zero weight decay strictly degrades performance. This is consistent with the broader picture of the Parameter Golf leaderboard, where strong submissions have collectively converged on a very similar optimiser recipe; the implication is that further gains under the same training recipe are unlikely to come from optimiser-side retuning, and we therefore inherit the baseline optimiser unchanged and concentrate the remaining compute budget on architectural and quantization modifications.

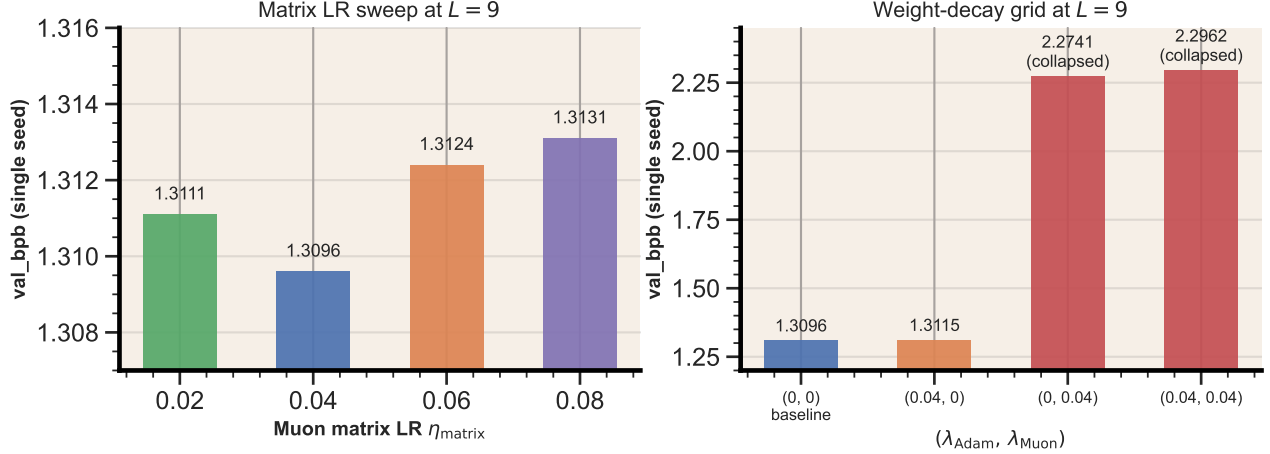


Figure 20: Baseline optimizer hyperparameter ablations at $L = 9$, single seed. Left: Muon matrix-LR sweep — the baseline default $\eta_{\text{matrix}} = 0.04$ is the optimum. Right: weight-decay grid — any non-zero weight decay on the Muon-managed parameter group crushes the model (BPP jumps to ~ 2.28 ; artifact size shrinks to < 1 MB, indicating weight collapse).

C.7 Parallel Residuals

Both routing parameters are initialised symmetrically so that the two lanes start identical and the model must *earn* any divergence during training.

Formally, attention reads from $\ell^{(0)}$ and MLP from $\ell^{(1)}$:

$$a = s_{\text{attn}} \odot \text{Attn}(\text{RMSNorm}(\tilde{\ell}^{(0)})), \quad \ell^{(j)} \leftarrow \Lambda_{\ell,0}^{\text{resid}} \ell^{(j)} + \Lambda_{\ell,0,j}^{\text{post}} a, \quad j \in \{0,1\}, \quad (1)$$

$$m = s_{\text{mlp}} \odot \text{MLP}(\text{RMSNorm}(\tilde{\ell}^{(1)})), \quad \ell^{(j)} \leftarrow \Lambda_{\ell,1}^{\text{resid}} \ell^{(j)} + \Lambda_{\ell,1,j}^{\text{post}} m, \quad j \in \{0,1\}. \quad (2)$$

This is consistent with the *parallel residuals* formulation from [1]

Sweep methodology. At $L = 9$ we swept the parallel-start layer $\text{psl} \in \{-1, 0, 4, 7\}$ with a single seed (1337), where $\text{psl} = -1$ denotes the sequential-block control (equivalent to the baseline). The four configurations correspond to: no PR ($\text{psl} = -1$, 0 parallel layers), entire stack parallel ($\text{psl} = 0$, 9 parallel layers), entire decoder parallel ($\text{psl} = 4$, 5 parallel layers), and final two layers only ($\text{psl} = 7$, 2 parallel layers).

Results ($L = 9$). The single-seed sweep identifies $\text{psl} = 4$ as the optimum (Figure 21, Table 7). The U-shape is consistent: too few parallel layers ($\text{psl} = 7$) underuses the mechanism, while too many ($\text{psl} = 0$) destabilises the early encoder layers where sequential composition is more important.

psl	n_{parallel}	val_bpb (seed 1337)
-1	0	1.3085
7	2	1.3058
4	5	1.3044
0	9	1.3077

Table 7: PR psl sweep at $L = 9$, single seed.

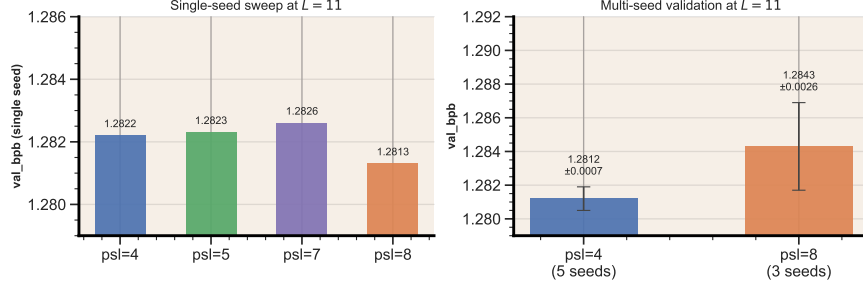


Figure 21: PR parallel-start-layer sweep at $L = 9$ (single seed). The optimum at $\text{psl} = 4$ corresponds to “entire decoder parallel”.

Asymmetric vs. symmetric routing initialisation. At $\text{psl} = 4$ we additionally tested asymmetric initialisation of the routing matrix, where the cross-lane entries are zeroed at init ($\Lambda_{\ell,0,1}^{\text{post}} = \Lambda_{\ell,1,0}^{\text{post}} = 0$ at step 0), forcing the two lanes to start fully separated. Single-seed results agree within noise (1.3046 vs. 1.3042), and the 3-seed validation gives 1.3058 ± 0.0021 (asymmetric) vs. 1.3057 ± 0.0016 (symmetric), a difference of $+0.0002$ BPB. We retain symmetric initialisation as the default for minimal inductive bias.

Multi-seed validation. A 3-seed validation at the single-seed optimum $\text{psl} = 4$ confirms the gain. Mean $\pm$ std across seeds $\{42, 123, 1337\}$: 1.3057 ± 0.0016 BPB, a -0.0044 BPB improvement (3.4σ of baseline noise; Figure 22).

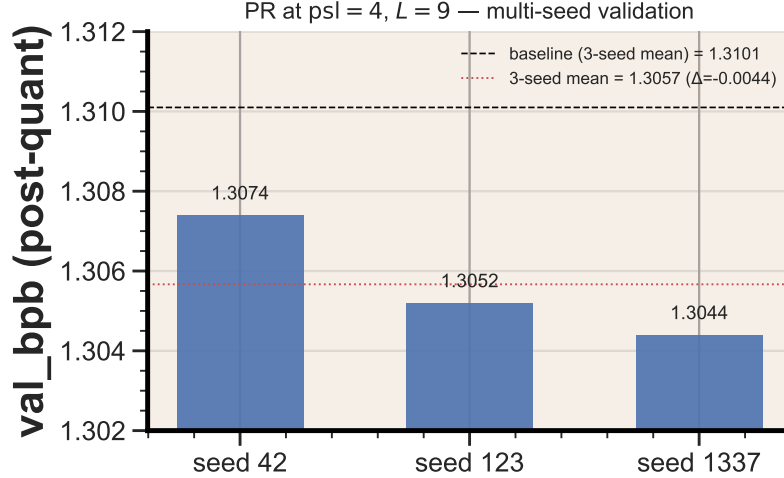


Figure 22: PR at $\text{psl} = 4$, $L = 9$: per-seed validation BPB. Dashed line marks the 3-seed baseline mean (1.3101); the -0.0044 BPB improvement holds across all three seeds.

Transfer to $L = 11$. We re-swept psl at $L = 11$ over the same logical range ($\text{psl} \in \{4, 5, 7, 8\}$, single seed) and again validated the single-seed winner across multiple seeds (Figure 23). The single-seed result misleadingly favoured $\text{psl} = 8$ (1.2813 BPB), but the 3-seed mean lands at 1.2843 ± 0.0026 — the seed-1337 result was a -1.2σ favourable outlier. By contrast, the 5-seed validation at $\text{psl} = 4$ gives 1.2812 ± 0.0007 , both lower in mean BPB and $3.7\times$ tighter in std. We adopt $\text{psl} = 4$ as the final configuration.

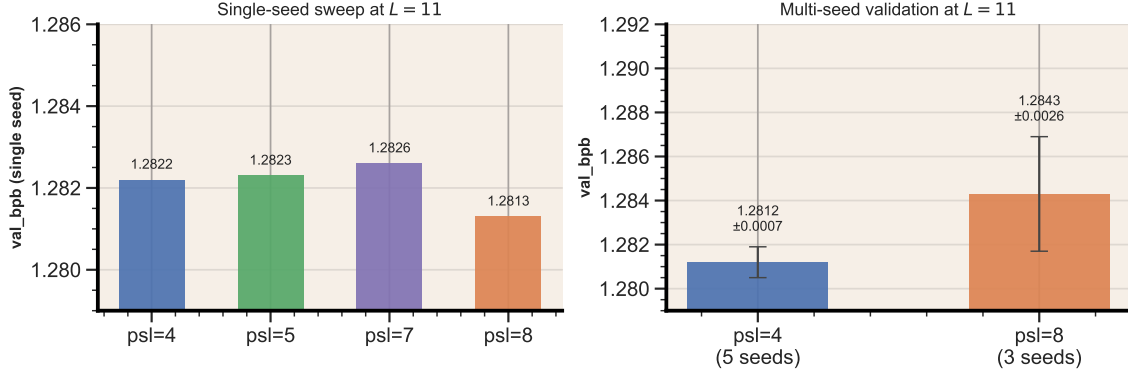


Figure 23: Left: single-seed psl sweep at $L = 11$, suggesting psl = 8 as the winner. Right: 3-seed multi-seed validation reveals that the seed-1337 result for psl = 8 was a favourable outlier; psl = 4 has both lower mean BPB and substantially tighter seed variance.

C.8 Depth Recurrence

Sweep methodology. We pursued a staged search at $L = 9$: (i) which contiguous set of layers to recur, (ii) which sub-component (attention only, MLP only, or both) to re-apply, and (iii) how many layers to recur (the “horizon” sweep). Each step was single-seed (seed 1337); the final configuration was re-validated with three seeds.

Position sweep ($L = 9$). We first swept short contiguous sets $\mathcal{R} \in \{[4], [4, 5], [3, 4], [5, 6]\}$ at the encoder-decoder hinge with whole-block target. The inner-hinge configuration $\mathcal{R} = [3, 4]$ emerged as the single-seed optimum (Figure 24, left panel).

Horizon sweep at target=both. We then extended the contiguous set with `recur_target=both`: $\mathcal{R} \in \{[3, 4, 5], [2, 3, 4, 5], [3, 4, 5, 6]\}$. The 4-layer extension $\mathcal{R} = [2, 3, 4, 5]$ achieves the lowest single-seed BPB (Figure 24, right panel) and was carried forward as the headline DR configuration.

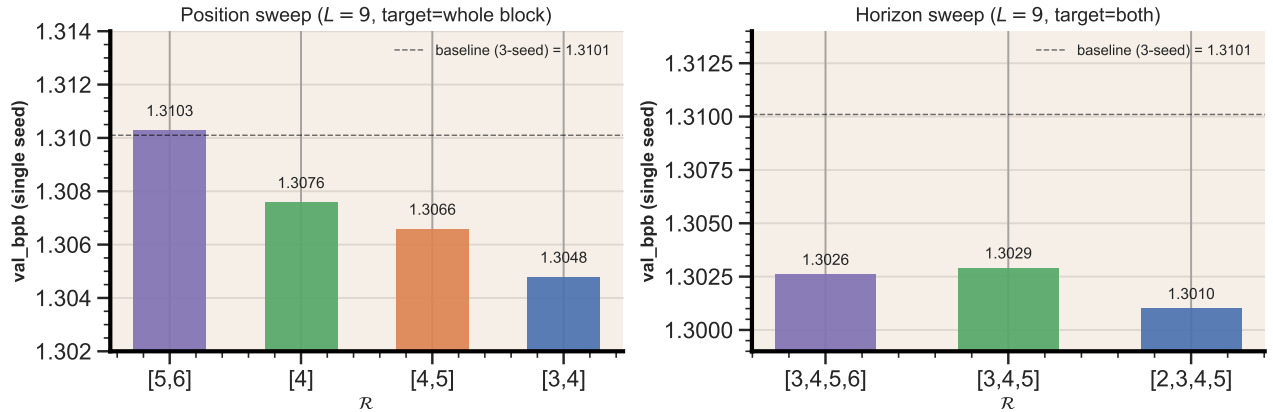


Figure 24: DR sweep at $L = 9$ (single seed). Left: short position sweep over 1–2 contiguous layers at `recur_target=whole-block`; the inner-hinge configuration $\mathcal{R} = [3, 4]$ wins. Right: horizon extension at `recur_target=both`; the 4-layer extension $\mathcal{R} = [2, 3, 4, 5]$ achieves the lowest single-seed BPB. Dashed line marks the 3-seed baseline mean.

Sub-component ablation. At $\mathcal{R} = [4, 5]$ we tested whether to re-run only the attention sublayer, only the MLP sublayer, or both. `recur_target=both` outperforms either single-component variant (Figure 25); we attribute this to compounded refinement of *both* attention-side routing and MLP-side feature transformation within the recurrent block.

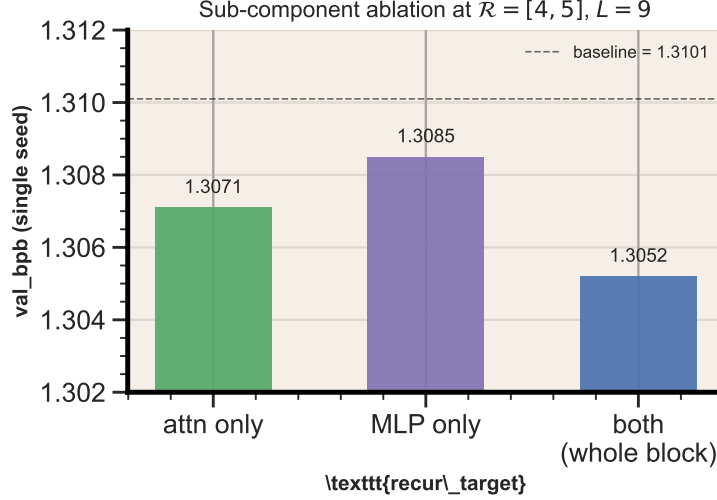


Figure 25: Sub-component ablation at $\mathcal{R} = [4, 5]$, $L = 9$, single seed. Re-running the whole block outperforms re-running only the attention or only the MLP sub-layer.

Multi-seed validation ($L = 9$). A 3-seed validation at the horizon-sweep optimum $\mathcal{R} = [2, 3, 4, 5]$, `recur_target=both` confirms the gain: 1.3022 ± 0.0029 BPB, a -0.0079 BPB improvement (6.1σ of baseline noise). This is the largest single-mechanism contribution among the three architectural additions, despite DR adding zero parameters.

Transfer to $L = 11$. At $L = 11$ with the mixed-bitwidth INT6/INT8 quantization scheme of Section 4, we re-ablated the recurrent set. Both $\mathcal{R} = [3, 4, 5]$ and $\mathcal{R} = [2, 3, 4, 5]$ are statistically indistinguishable on BPB (Figure 26, left: 1.2812 ± 0.0007 vs. 1.2809 ± 0.0022 , $\Delta = +0.0003$ BPB, 0.5σ pooled). However, $\mathcal{R} = [3, 4, 5]$ has a $3.7\times$ tighter seed variance *and* a 0.46 MB smaller compressed artifact (Figure 26, right), since one fewer layer requires the higher-precision INT8 storage mandated by the mixed-bitwidth scheme. We therefore adopt $\mathcal{R} = [3, 4, 5]$ as the final configuration.

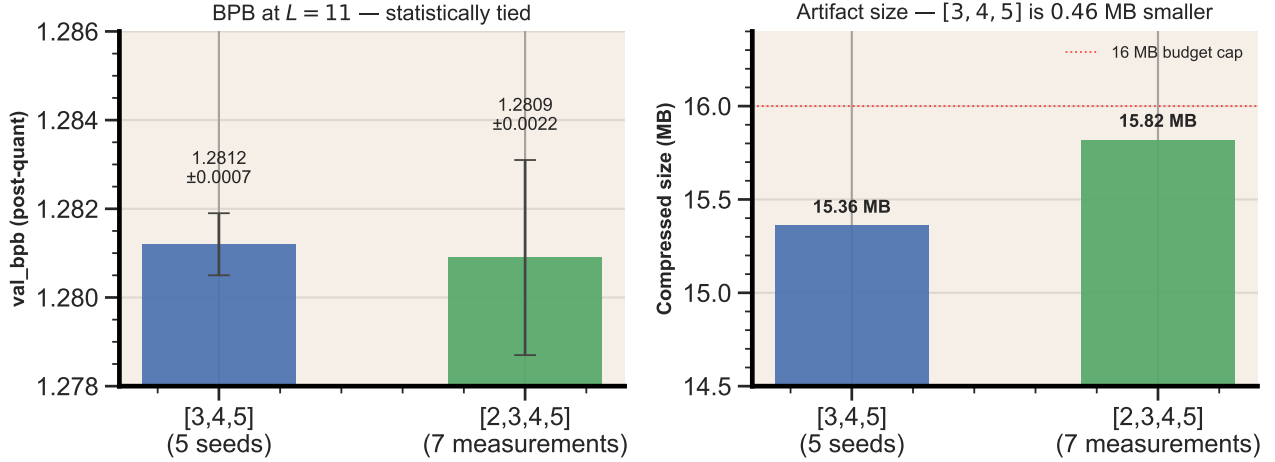


Figure 26: DR re-ablation at $L = 11$, head-to-head between $\mathcal{R} = [3, 4, 5]$ and $\mathcal{R} = [2, 3, 4, 5]$. Left: post-quantization BPB — statistically tied. Right: compressed artifact size — $\mathcal{R} = [3, 4, 5]$ is 0.46 MB smaller because one fewer layer requires INT8 storage under the mixed-bitwidth scheme. Combined with its $3.7\times$ tighter seed variance, $\mathcal{R} = [3, 4, 5]$ is the safer final choice.

C.9 Attention Output Gate

On top of the standard formulation we make two deliberate design choices: (i) W_g is initialised to zero so that $g = 2\sigma(0) = 1$ at the start of training and the gate is *transparent* (the block degenerates to the baseline at iteration 0); (ii) the activation is $2\sigma(\cdot) \in (0, 2)$ rather than plain sigmoid, so the gate can both *attenuate* ($g < 1$) and *amplify* ($g > 1$) a head. The gate adds only $w_g \cdot H = 12 \cdot 8 = 96$ parameters per attention

sublayer.

Sweep methodology. We swept two axes: (i) the source of the gate input (pre-projection query features vs. post-projection attention features), and (ii) the gate input width $w_g \in \{6, 8, 10, 12, 24\}$. All entries are single-seed (1337) unless explicitly noted as multi-seed.

Source ablation. At $w_g = 12$, the post-projection source ($\text{src}=\text{proj}$) is uniformly beneficial, while the pre-projection query-feature source ($\text{src}=\text{q}$) slightly *regresses* relative to the disabled control (Table 8). We retain $\text{src}=\text{proj}$ for all subsequent ablations.

configuration	val_bpb (seed 1337)	Δ vs disabled
disabled (control)	1.3098	0.0000
$\text{src}=\text{q}, w_g = 12$	1.3115	+0.0017 (worse)
$\text{src}=\text{proj}, w_g = 12$	1.3052	−0.0046

Table 8: Gate source ablation at $w_g = 12$, $L = 9$ (single seed).

Width sweep at $\text{src}=\text{proj}$ ($L = 9$). Figure 27 shows the single-seed width sweep. $w_g = 8$ achieves the lowest single-seed BPB (-0.0058 vs. the disabled control), but the width curve is shallow: $w_g \in \{8, 12\}$ are within 0.0012 BPB of each other.

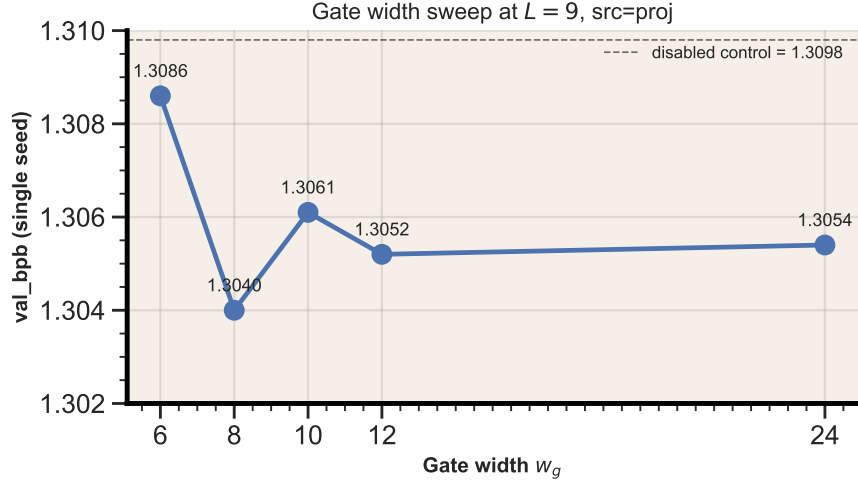


Figure 27: Gate width sweep at $L = 9$, $\text{src}=\text{proj}$, single seed. The curve is shallow between $w_g = 8$ and $w_g = 12$; both are candidates for the headline configuration.

Multi-seed validation: standalone vs. composition. Both $w_g = 8$ and $w_g = 12$ were validated with three seeds at $L = 9$. *Standalone*, the narrower variant $w_g = 8$ has a marginally better mean (1.3042 ± 0.0004 vs. 1.3058 ± 0.0008 , $\Delta = -0.0016$ BPB; Figure 28 left). However, the choice flips when the gate is composed with PR and DR: the full PR+DR+Gate($w_g = 12$) stack reaches 1.2927 ± 0.0007 BPB, versus 1.2935 ± 0.0015 for PR+DR+Gate($w_g = 8$) — Figure 28 right. We attribute this to the wider gate having more capacity to adjust per-head contributions in concert with PR’s two-lane routing. Since the parameter cost of $w_g = 12$ vs. $w_g = 8$ is only 32 floats per attention sublayer (negligible against the $\sim 1\text{M}$ parameters of a single block), we adopt $w_g = 12$ as the final choice for compositional reasons.

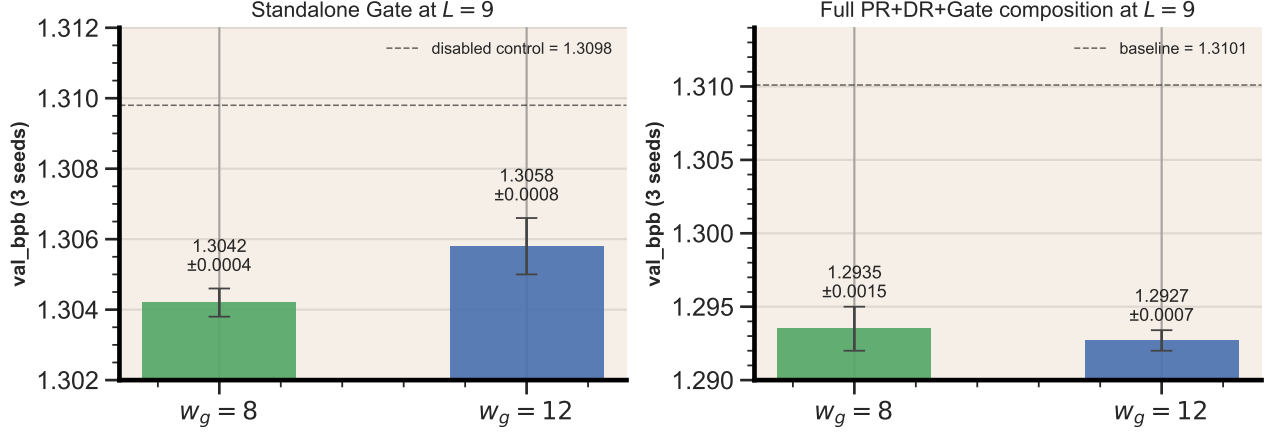


Figure 28: Standalone vs. composition multi-seed comparison of $w_g = 8$ and $w_g = 12$. Left: standalone Gate at $L = 9$ — $w_g = 8$ is marginally better. Right: full PR + DR + Gate composition at $L = 9$ — the wider $w_g = 12$ wins by -0.0008 BPB. The standalone winner is NOT the composition winner; we adopt $w_g = 12$ for the final stack. All values are 3-seed means ± 1 std.

C.10 Internal dynamics of the architectural additions

Parallel residuals. Figure 29 shows the per-layer 2×2 routing matrix at the end of training across three seeds: in every run the off-diagonal entries collapse and the attention/MLP outputs become near-orthogonal in their write targets. Figure 30 plots the mean λ_{resid} for the attention and MLP lanes over training; both remain in a stable range, indicating neither lane is gated off and that the mechanism is well-behaved end-to-end. The asym-vs-sym init comparison (Fig. 31) directly tests whether the endpoint specialisation alone suffices: pre-loading the lane-diagonal solution slows optimisation enough that the symmetric init wins at the 5 000-step budget, isolating the symmetry-breaking phase as load-bearing.

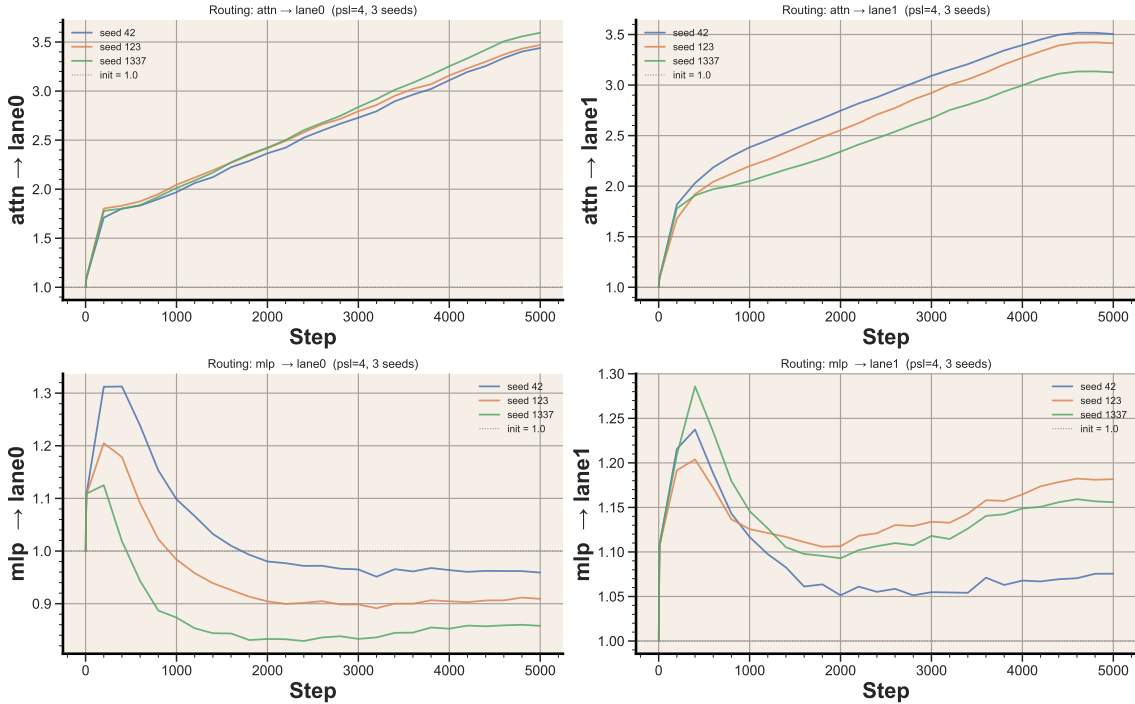


Figure 29: Routing-matrix trajectories for the psl=4 configuration across three seeds. Off-diagonal entries (attn $\rightarrow$ lane1, mlp $\rightarrow$ lane0) collapse in every seed, while the diagonal entries are retained: attention and MLP develop non-overlapping write channels into the residual stream.

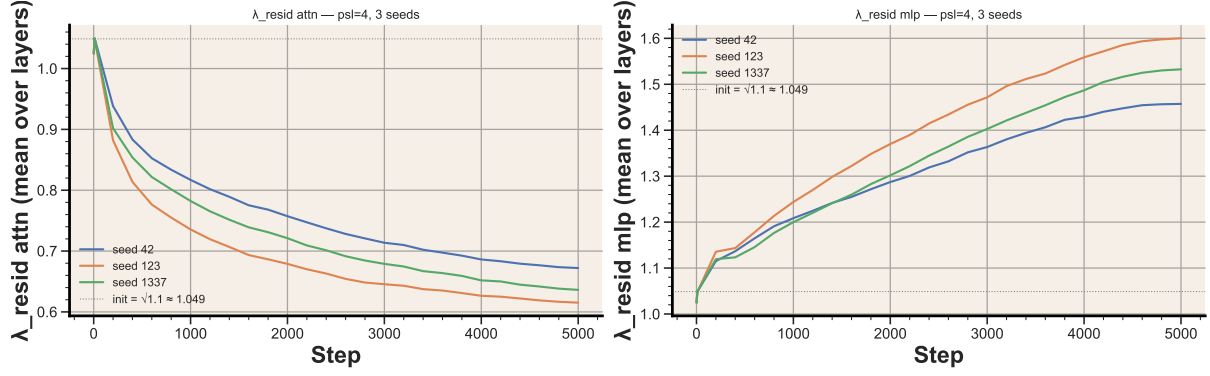


Figure 30: Layer-mean λ_{resid} for the attention and MLP lanes over training ($\text{psl}=4$, three seeds). Both scalars remain bounded around their $\sqrt{1.1}$ initialisation, confirming that neither lane is gated off during training.

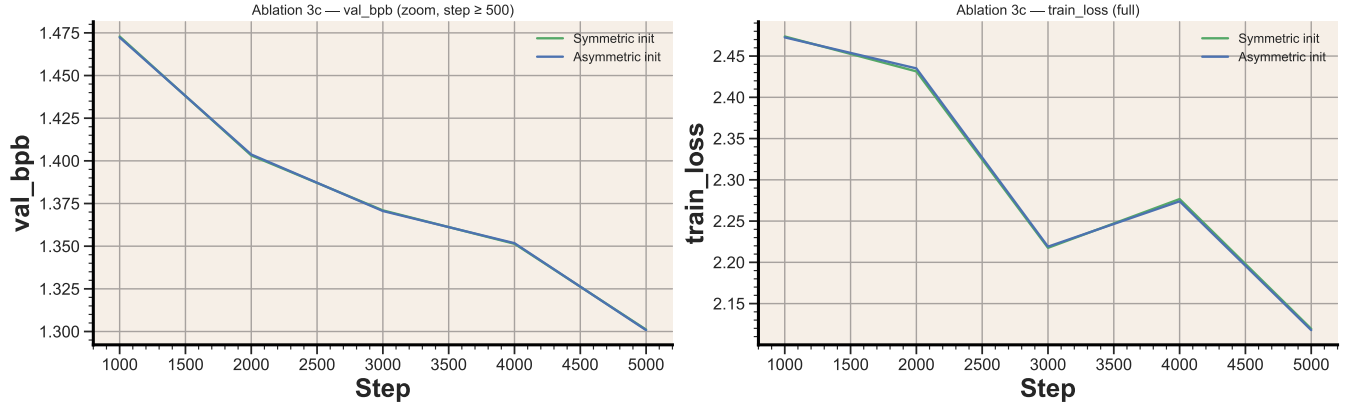


Figure 31: Symmetric vs. asymmetric routing initialisation at $\text{psl}=4$, matched compute (5 000 steps, seed 1337). Pre-loading the lane-diagonal asymptote as the init slows optimisation enough that the symmetric init wins on final val_bpb , isolating the symmetry-breaking phase rather than the endpoint as the load-bearing piece.

Depth recurrence. The sub-component sweep at $L=9$ (Fig. 32) and the head-to-head at $L=11$ (Fig. 33) show that the bpb gain saturates quickly with recurrence depth and is concentrated in mid-stack layers, consistent with DR acting as effective depth rather than capacity.

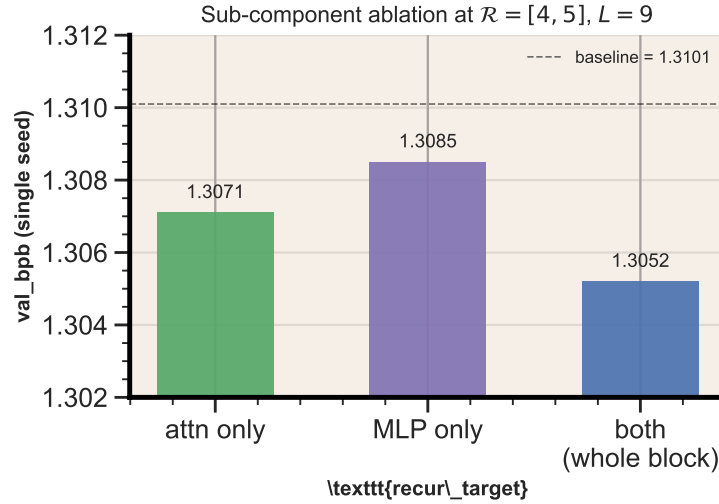


Figure 32: Depth-recurrence sub-component sweep at $L=9$: contribution of each recurrent sub-block to the final val_bpb .

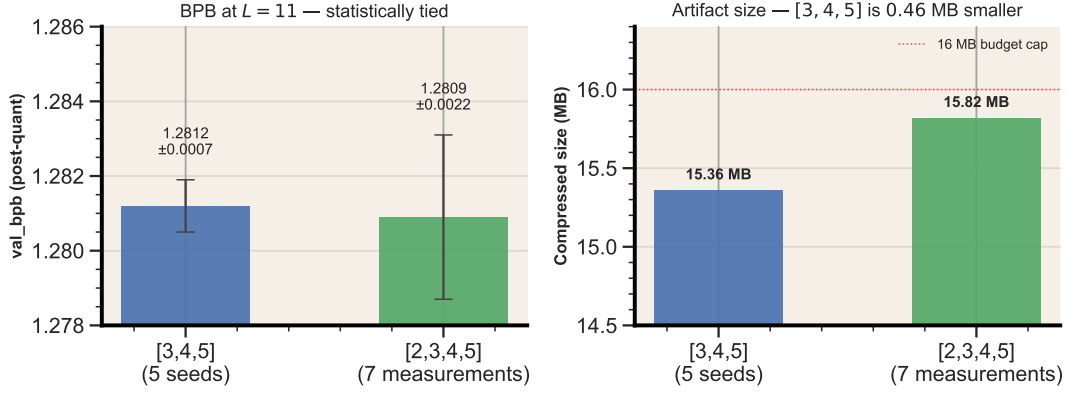


Figure 33: Head-to-head comparison of depth recurrence variants at $L=11$; the gain saturates quickly with recurrence depth.

Attention gate. The gate-width sweep at $L=9$ (Fig. 34) and the 8-vs-12 head-to-head (Fig. 38) show a clear interior optimum, which is what we would expect if the gate is filtering attention rather than just adding parameters.

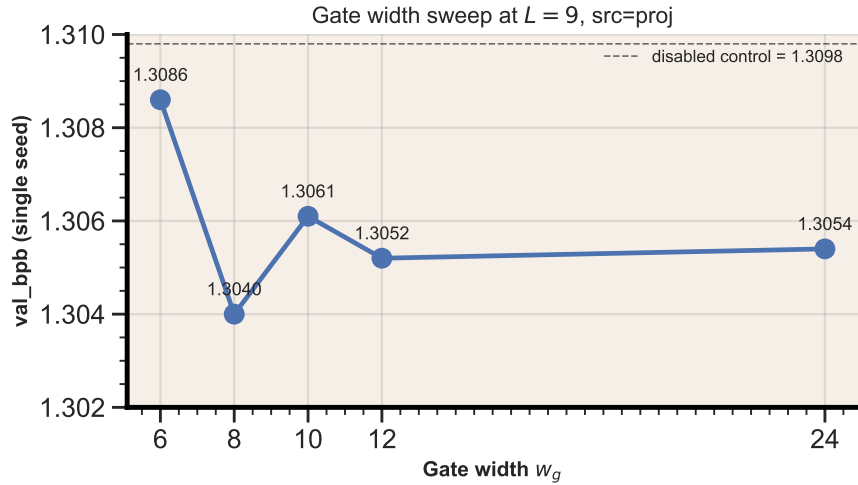


Figure 34: Attention-gate width sweep at $L=9$: val_bpb exhibits a clear interior optimum, ruling out a pure parameter-count explanation for the gate's effect.

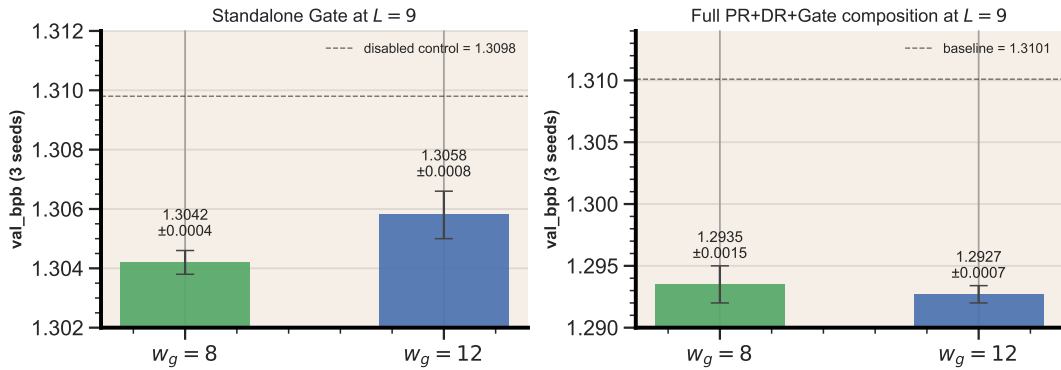


Figure 35: Head-to-head between attention-gate widths 8 and 12 at $L=9$, confirming the interior optimum of Fig. 34.

C.11 Exclusive Self-Attention (XSA)

Exclusive Self-Attention [14] is a parameter-free modification of self-attention motivated by an observed similarity bias: the per-head SDPA output y_i tends to be strongly aligned with the token’s own value vector v_i , so attention spends capacity on a pointwise transformation that the residual stream and the following MLP can already perform. XSA removes the component of y_i along v_i ,

$$y_i \leftarrow y_i - (y_i^\top \hat{v}_i) \hat{v}_i, \quad \hat{v}_i = v_i / \|v_i\|_2,$$

forcing each head to carry only information *orthogonal* to its self-value. The modification adds no parameters and costs only $\sim 5\%$ extra per step; the paper reports consistent gains on language modelling up to 2.7B parameters, with the gap growing with sequence length.

We evaluated XSA at two levels in our setting. Standalone, on top of the 9-layer baseline (3 seeds), it improves the quantised validation BPB by -0.0033 (1.3068 ± 0.0011 vs. the baseline’s 1.3101 ± 0.0013 , $\sim 2.5\sigma$); the smallest standalone gain among the mechanisms we tried, but the only parameter-free one. Stacked on top of our headline composition (parallel residuals + depth recurrence + smeared attention gate), however, XSA does *not* compose: the pre-quantization Δ is $+0.0001$ bpb and the post-quantization Δ is $+0.0005$ bpb, both well within seed noise, and the composition’s seed standard deviation grows roughly $4\times$ (from 0.0007 to 0.0030). We read this as XSA’s per-head output shaping being already captured by the smeared attention gate’s per-head modulation and by the parallel-residual two-lane routing; given the extra compute and the increased seed variance, we therefore exclude XSA from the final architecture.

C.12 Attention Residuals

Standard residual connections aggregate information across depth using fixed unit weights. If h_l denotes the hidden state entering layer l and f_l the corresponding layer transformation, the usual residual recurrence is

$$h_l = h_{l-1} + f_{l-1}(h_{l-1})$$

Unrolling this recurrence shows that each layer receives an unweighted sum of all preceding layer outputs. Attention Residuals replace this fixed accumulation with a learned, content-dependent attention mechanism over the depth dimension, allowing each layer to selectively retrieve earlier representations rather than relying only on the immediately preceding residual stream.

For a model with token embedding h_1 , define the value sources

$$v_i = \begin{cases} h_1, & i = 0, \\ f_i(h_i), & 1 \leq i \leq l-1 \end{cases}$$

The input to layer l is then computed as

$$h_l = \sum_{i=0}^{l-1} \alpha_{i \rightarrow l} v_i, \quad \sum_{i=0}^{l-1} \alpha_{i \rightarrow l} = 1$$

The attention weights are obtained by applying a softmax over previous depth sources:

$$\alpha_{i \rightarrow l} = \frac{\phi(q_l, k_i)}{\sum_{j=0}^{l-1} \phi(q_l, k_j)},$$

where the paper uses $\phi(q, k) = \exp(q^\top \text{RMSNorm}(k))$ to obtain the usual softmax. In the full Attention Residuals formulation, the query is a learned layer-specific pseudo-query, namely $q_l = w_l \in \mathbb{R}^d$, and the keys are directly tied to the values, namely $k_i = v_i$. Thus each layer learns how strongly to attend to the embedding and to each previous layer output.

Block Attention Residuals. Full Attention Residuals require access to all previous layer outputs, giving memory cost proportional to $O(Ld)$ for depth L and hidden dimension d . Block Attention Residuals reduce this cost by partitioning the network into N contiguous blocks. Within each block, layer outputs are accumulated into a single block representation,

$$b_n = \sum_{j \in B_n} f_j(h_j),$$

where B_n denotes the set of layers in block n . Cross-depth attention is then performed over the token embedding, previous block representations, and the current block’s partial sum, rather than over every individual preceding layer. For a layer inside block n , the source set is approximately

$$\{b_0, b_1, \dots, b_{n-1}, b_n^{i-1}\},$$

where $b_0 = h_1$ and b_n^{i-1} is the partial residual accumulated so far inside the current block.

Skip Attention Residuals I created a simpler version of attention residuals that only replaces the U-net like skip connections with an attention-style aggregation, while leaving the internal transformer residual stream unchanged. Therefore, I remove weighted fixed skip connections and instead resort to an attention mechanism to reweight all inputs from the skip stack. Conceptually, for decoder block i , I substitute:

$$x \leftarrow x + w_i \odot \text{skip}_i$$

with:

$$V = [\text{skip}_1, \dots, \text{skip}_k, x], \quad K = \text{RMSNorm}(V), \quad \alpha = \text{softmax}(q^\top K)$$

$$x \leftarrow \sum_j \alpha_j V_j$$

where q is the learned vector from the block's projection.

The hierarchy of val-BpB losses for the 3 variants is as follows: Skip-Attention, namely the most parameter-efficient variant that preserves residual mixing, performs best achieving a consistent mean reduction of 0.036 in val BpB compared to the baseline. In second spot, there is the full attention residual variant which achieves a val bpb on average of 1.328, which is higher than the baseline by 0.018 and, in the last spot, the block attention residuals which significantly model performance bringing the val bpb up to 1.46. We believe this is probably due to the fact that the model is small and it requires more training steps to learn the correct routing of residuals although it could possibly outperform the baseline if trained for longer. Still, the skip attention variant which requires intuitively less residual routing learning from the model performs better than the baseline thanks to its higher flexibility.

Nonetheless, when combined with the other architectural design choices, skip attention seems to cause significant increase in val-bpb loss up to 1.35 suggesting its incompatibility with the latter for the relatively small GPT model analysed.

The main results of the experiments performed are displayed below.

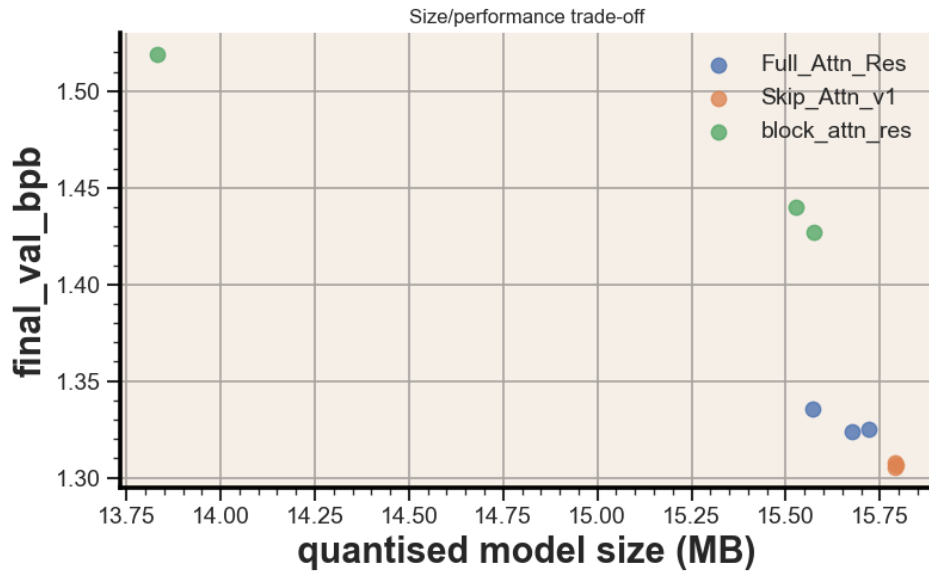


Figure 36: Post quantization (as in baseline) val-bpb performance vs memory size scatter plot showing trade-off of different Attention Residual variants



Figure 37: Barplot of post-Quantization mean final val-bpb loss for different attention residual variants

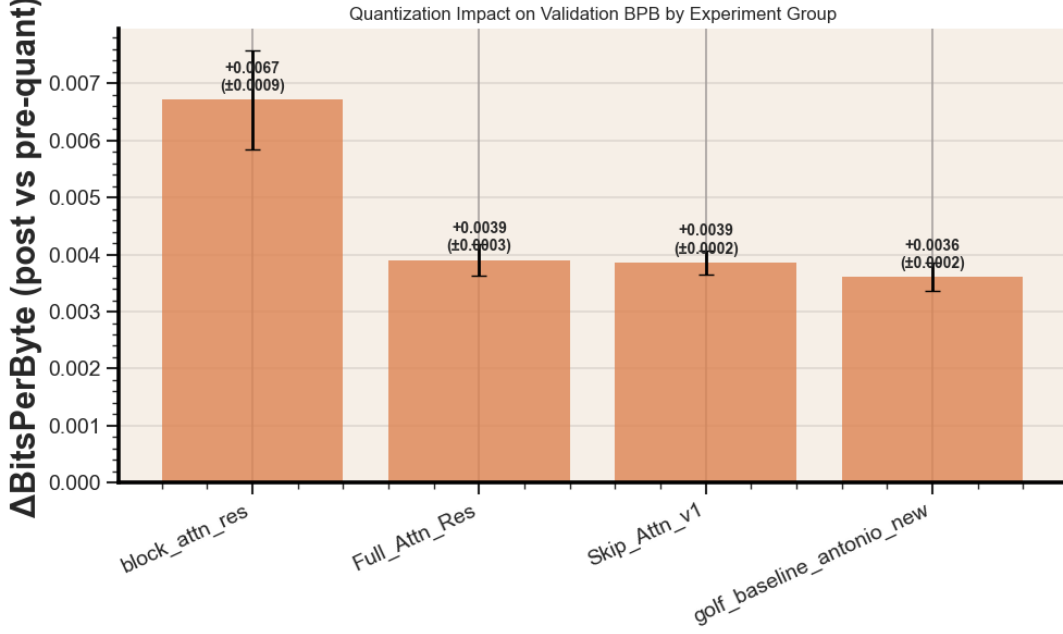


Figure 38: difference in val-BpB loss of different attention residual variants before and after quantization (as in baseline)

C.13 Quantization of Language Models

Applying quantization to neural networks helps by reducing their memory footprint by using numerical formats of a lower precision. There are two main quantization approaches:

1. Post-training Quantization: Weights are quantized after training.
2. Quantization Aware Training: Weights are quantized and immediately dequantized during training so that the model learns to be robust to quantization errors.

The baseline model, as described before, relies only on post-training quantization, in particular on classical integer quantization. Classical integer quantization is a technique that converts higher precision floating point tensors (like FP32 or FP16) to lower precision integers (like INT8 or INT4) by using the following two parameters:

1. Scale (S): A floating point value that represents the step size between each integer level.
2. Zero Point (Z): The integer value that corresponds to the real world floating point value 0.0 in the original dataset.

We define the mapping from a floating point x to a quantized integer q as:

$$q = \text{round} \left(\frac{x}{S} \right) + Z$$

Prior work has shown that integer-only quantization reduced inference cost while preserving accuracy (if scaling and calibration are chosen carefully) [6]. In the setting of large language models, post training quantization appears to be a very important technique because model size directly affects deployment cost and memory feasibility.

C.14 Uniform quantization design study

In the basic quantization design study, we evaluate post-training weight quantization choices along: granularity, symmetry and calibration. For a given floating-point tensor W , the uniform symmetric quantization maps each value to an integer code:

$$q = \text{clip} \left(\text{round} \left(\frac{W}{s} \right), q_{\min}, q_{\max} \right),$$

followed by dequantization

$$\hat{W} = sq.$$

For b -bit symmetric quantization, we used

$$q_{\min} = -(2^{b-1} - 1), \quad q_{\max} = 2^{b-1} - 1.$$

Calibration was tested in two firstly in an uncalibrated setting using the raw tensor range, and then in a percentile-clipped setting that clips extreme values before computing the quantization scale. Each configuration was trained with the same training schedule and then evaluated after a full quantize -> compress -> decompress -> dequantize pipeline.

C.15 NormalFloat-4 (NF4)

We have additionally implemented the NF4 codebook quantization technique as a more advanced 4-bit method. Unlike uniform INT4, where we assign evenly spaced integer levels, here we use a non-uniform set of representable values designed to better match normally distributed weights. For each tensor or output channel, we normalize the weights using a factor of scale and we assign them to the nearest NF4 codebook entry. These weights are then converted to 4-bit indices depending on the Gaussian bin they fall closest to, these 4-bit numbers is what we actually store in the GPU memory. When the model needs to perform computations (like matrix multiplications), we map the 4-bit indices into their approximate floating-point values using a pre-defined NF4 codebook and the block scale factor. The computations are then performed in the higher mapped precision (like 16-bit or 32-bit) before being stored again.

C.16 GPTQ

GPTQ is a one-shot, post-training weight quantization method. Unlike naive round-to-nearest, which rounds each weight independently, GPTQ rounds weights sequentially and compensates for each rounding error by adjusting the not-yet-quantized weights, using second-order information from a small calibration set. This enables aggressive bit-widths (int4-int6) with far less accuracy loss than naive rounding.

Consider a linear layer $Y = WX$ with weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and calibration activations X . Quantizing W to $\widehat{W}$ perturbs the output by $\|(W - \widehat{W})X\|_2^2$, a quadratic objective whose curvature is the Hessian $H = XX^\top$. GPTQ quantizes the columns of W left to right; after rounding column q , it distributes the error to the remaining columns $j > q$ via

$$W_{:,j} \leftarrow W_{:,j} - (W_{:,q} - \widehat{W}_{:,q}) \frac{[H^{-1}]_{qj}}{[H^{-1}]_{qq}}. \quad (3)$$

The coefficient $[H^{-1}]_{qj}/[H^{-1}]_{qq}$ is the optimal least-squares adjustment: strongly coupled columns receive a large correction, independent ones little. By the final column, earlier errors have been partially absorbed, so the total output error is much smaller than under independent rounding. A small diagonal dampening $H \leftarrow H + \lambda dI$ ensures H is invertible, and columns are processed in blocks (via the Cholesky factor of H^{-1}) for efficiency.

GPTQ runs once after training, needs only forward passes for the Hessian (no backpropagation), and costs $O(d_{\text{in}}^3)$ per layer—negligible at our dimensions. In our experiments its advantage over naive rounding was minimal at int8 but grew at lower bit-widths, where the larger rounding errors give the compensation more to correct.

C.17 AWQ

AWQ is based on the idea there is a small fraction (0.1%-1%) of salient channels. Skipping the quantization of these salient channels will significantly reduce the quantization loss. The insight to find the salient weight channels is that we should refer to the activation distribution instead of the weight distribution, despite we are doing weight only quantization; this works as weight channels corresponding to larger activation magnitudes are more salient since they process more important features.

The technique works by first running a small calibration set through the model and for each linear layer record the average activation magnitude $s_X = \mathbb{E}(|X|)$; AWQ parameterizes a channel-wise scaling vector as $s = s_X^\alpha$ and searches for the exponent α that minimizes the reconstruction error after quantization: $\alpha^* = \arg \min_\alpha L(s_X^\alpha)$ where $L(s) = \|Q(W \cdot \text{diag}(s)) (\text{diag}(s)^{-1} X) - WX\|$. Since

$$WX = (W \text{diag}(s)) (\text{diag}(s)^{-1} X),$$

the scaling operation leaves the original computation unchanged. After finding the optimal scaling vector the weights are rescaled and quantized as

$$W_q = Q(W \text{diag}(s^*)).$$

During inference, the corresponding activations are inversely scaled by $\text{diag}(s^*)^{-1}$.

This effectively allocates more quantization precision to salient channels, reducing quantization error without changing the model's output. Then scale the weights: $W_{\text{scaled}} = W \cdot \text{diag}(s^*)$ and quantize the scaled weights $Q(W_{\text{scaled}})$. Then during inference we compensate by inversely scaling the activations, $\text{diag}(s^*)^{-1} X$.

C.18 LQER

LQER (Low-rank Quantization Error Reconstruction) complements quantization by storing a compact correction for the residual error, rather than trying to make the rounding itself more accurate. After a weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is quantized to $\widehat{W}$, the quantization error $E = W - \widehat{W}$ is approximated by a low-rank factorization and added back at inference time, so that $W \approx \widehat{W} + AB$.

The factors are obtained from the truncated singular value decomposition of the error. Writing $E = U\Sigma V^\top$ and keeping the top k components,

$$A = U_{:,1:k} \Sigma_{1:k}, \quad B = V_{1:k,:}^\top, \quad A \in \mathbb{R}^{d_{\text{out}} \times k}, \quad B \in \mathbb{R}^{k \times d_{\text{in}}}, \quad (4)$$

so the correction AB captures the dominant directions of the error at a storage cost of only $k(d_{\text{out}} + d_{\text{in}})$ values—a small fraction of the full matrix for low rank k (we use $k = 4$). In the activation-aware variant (L²QER), the error is first scaled by the per-channel activation magnitudes before the SVD, so that the correction prioritizes the directions that most affect the layer output; the scaling is undone in B to preserve equivalence.

Because the correction adds storage, LQER is applied selectively to the most sensitive layers only. We rank layers by the activation-weighted reconstruction error $\text{tr}(E^\top HE)$, where $H = XX^\top$, and apply the rank- k correction to the top- K layers (we use $K = 3$). LQER is fully post-training and composes naturally with GPTQ: GPTQ first minimizes the rounding error, and LQER then recovers the dominant residual that remains, trading a small increase in artifact size for reduced quantization damage.

C.19 Quantization Aware Training

Quantization-aware training (QAT) simulates quantization during the forward pass, so the optimizer is driven toward weights that are robust to it. Each weight matrix is passed through a *fake quantization* that rounds and immediately dequantizes it. For a symmetric per-row b -bit scheme with $m = 2^{b-1} - 1$ and scale $s = \max_j |W_{ij}|/m$,

$$\widehat{W} = s \cdot \text{clip}(\text{round}(W/s), -m, m). \quad (5)$$

Since rounding is non-differentiable, gradients are propagated with the *straight-through estimator* (STE), implemented as $W_{\text{fake}} = W + (\widehat{W} - W)_{\text{detach}}$: the forward pass uses the rounded weights, while the backward pass treats rounding as the identity and updates the full-precision master weights, which gradually drift toward values that quantize with minimal loss.

To be effective, the fake quantization mirrors the deployment scheme per layer: non-recurrent layers are quantized at int6, recurrent layers at int8 (their weights are reused within a forward pass, so rounding errors would otherwise compound), and the tied embedding at int8. We use *late* QAT, enabling fake quantization only after a fraction $\tau = 0.25$ of training, so the model first converges in full precision and then adapts to the grid; the model is recompiled at activation so the fake-quantization branch is actually traced. QAT adds only element-wise operations and leaves the backward pass unchanged, so its overhead is negligible; its sole purpose is to shrink the gap between the full-precision and quantized models.

C.20 Quantization Ablations

In this section we described the results of the quantization ablations:

C.20.1 bidwidth ablation

We tested three different bidwidths for the baseline quantization setup. As shown in Figure 6, using int6 saves 3.82 MB over int8, increasing the *val_bpb* of just +0.0575. The most efficient bitwidth at saving space is int4 which compresses the serialized dimension to 7.83 MB (Figure 7). However, int4 increases the *val_bpb* to 2.2457 on average over three seeds, leading therefore to a much worse performance. For this reason, the best choice to get more space and not degradate the accuracy is int6.

C.20.2 Per-channel vs per-tensor ablation

We tried to analyze the impact of scaling at different bidwidths. Over all the basic quantization sweeps reported in Table2, the best performance was gotten with the *8-bit per-channel symmetric uncalibrated quantization* configuration, which achieved a final validation BPB of 1.3101 ± 0.0011 . This was nearly lossless relative to the pre-quantized model, increasing BPB metric only by +0.0038.

Among 4-bit uniform methods, the best configuration was 4-bit per-channel symmetric uncalibrated quantization, which achieved 2.5749 ± 0.0328 BPB. This result was however much worse than the best 8-bit setting but much better than per-tensor INT4 variants, showing that per-channel scaling is essential at low bitwidths.

C.20.3 NF4 ablation

The NF4 technique significantly improved over the best uniform INT4 baseline. The best NF4 result was observed for the *NF4 per-channel uncalibrated* configuration, with 1.8163 ± 0.0222 BPB, compared with 2.5749 ± 0.0328 BPB for the best uniform INT4 configuration. This gives an improvement of 0.7586 BPB over uniform INT4, while compressed size increases only from approximately 7.82 MB to 7.91 MB. Thus, NF4 substantially closes the quality gap between uniform INT4 and near-lossless INT8 while remaining in the same approximate storage regime as INT4. These results suggest that NF4 quantization is a stronger choice than unifrom INT4, given that per-channel normalization is used, as shown in Table 3.

C.21 Layer sensitivity analysis

We tested the impact on *val_bpb* of not quantizing:

1. the embeddings
2. the first transformer block
3. the last transformer block

4. attention scores of all transformer blocks
5. MLP parameters of all transformers blocks

As shown in Figure ??, the attention weights are the part that, if not quantized, increases the serialized file size the most. On the other hand, not quantizing the embeddings extremely improves the post-quantization *val_bpb*, without losing a lot of space.

It is also possible to see that the quantization of the MLP of all transformers blocks saves more than 12.5 MB of space, without affecting too drastically the average *val_bpb* over the seeds.

C.21.1 GPTQ ablation

As shown in Figure 9, GPTQ almost does not change the *val_bpb* for int8 and int6 bitwidths. The only granularity on which it has a positive result is int4, in which the baseline post-quantization *val_bpb* drops to 2.1511 from 2.2402.

C.21.2 GPTQ+LQER ablation

One of the most used quantization approaches in the leaderboard puts together GPTQ with int6, asymmetric LQER and int7 quantization for the embeddings. We tried to replicate this approach in our model, but we did not get significant improvements. As it is possible to see in Figure 10, the approach has a *val_bpb* of 1.362, in front of a file dimension of 13.85MB.

C.21.3 AWQ ablation

As last experiment, we tried to implement AWQ. As shown in figure 18, the obtained *val_bpb* with int4 is 1.853514, which is way better better than the *val_bpb* of the baseline with int4 but still not low enough to compete with the final setup we identified.